

DATA MINING

Prof. Roberta Siciliano

Prof. Giuseppe Longo

Prepared by:

Muhammad Sameer Kazi

D03000032

Hassan Muhammad Siraj Zafar

D03000033

Syed Muhammad Muneeb

D03000038



UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

Contents

1	Introduction	9
1.1	BACKGROUND INFORMATION	9
1.2	SIGNIFICANCE AND MOTIVATION	9
1.3	DATASET OVERVIEW	9
1.4	Tools and Technologies	10
1.4.1	HARDWARE	10
1.4.2	SOFTWARE	10
1.4.3	DEVELOPMENT SIDE TECHNOLOGIES	10
1.5	Architecture.....	10
1.5.1	Introduction	10
1.5.2	System architecture	10
1.5.3	Architecture model.....	11
1.6	Exploratory data analysis.....	11
2	Comparison of ML Models for Appendicitis Diagnosis	12
2.1	Random Forest (RF).....	12
2.1.1	Overview.....	12
2.1.2	Key Concepts.....	12
2.1.3	Algorithm Steps.....	12
2.1.4	Mathematical Representation.....	13
2.1.5	Advantages of Random Forest	13
2.1.6	Limitations.....	13
2.2	Support Vector Classification (SVC).....	13
2.2.1	Overview.....	13
2.2.2	Key Concepts.....	13
2.2.3	Algorithm Steps.....	13
2.2.4	Mathematical Representation.....	13
2.2.5	Advantages of SVC	14
2.2.6	Limitations.....	14
2.3	Logistic Regression (LR).....	14
2.3.1	Overview.....	14
2.3.2	Key Concepts.....	14
2.3.3	Algorithm Steps.....	14
2.3.4	Mathematical Representation.....	14
2.3.5	Advantages of Logistic Regression	14



2.3.6	Limitations.....	14
3	Libraries.....	15
3.1	Deep Learning & Computer Vision:	15
3.2	Data Processing & Transformation:	15
3.3	Model Training & Evaluation:.....	15
3.4	Visualization:	15
4	Functional and Data Description	16
4.1	Introduction	16
4.2	Data loading	16
4.3	Comprehensive Dataset Summary for Initial Exploration and Preprocessing	17
4.4	Assessing Missing Values in the Dataset	18
4.5	Data Cleaning:	20
4.5.1	Handling Missing Data by Removing Columns with Excessive Null Values	20
4.6	Variable Classification	22
4.6.1	Numerical variables	22
4.6.2	CATEGORICAL VARIABLES	23
5	Exploratory Data Analysis and Visualization.....	24
5.1	Correlation Heatmap:	24
5.1.1	Observation:	24
5.2	Box Plots:.....	26
5.2.1	Boxplot of Age	26
5.2.2	Boxplot of BMI (Body Mass Index).....	27
5.2.3	Boxplot of Height.....	27
5.2.4	Boxplot of Weight	27
5.2.5	Boxplot of Length_of_Stay.....	28
5.2.6	Boxplot of Alvarado_Score	28
5.2.7	Boxplot of Paediatric_Appendicitis_Score	29
5.2.8	Boxplot of Appendix_Diameter.....	29
5.2.9	Boxplot of Body_Temperature	30
5.2.10	Boxplot of WBC_Count (White Blood Cell Count).....	30
5.2.11	Boxplot of Neutrophil_Percentage.....	30
5.2.12	Boxplot of RBC_Count (Red Blood Cell Count).....	30
5.2.13	Boxplot of Hemoglobin	31
5.2.14	Boxplot of RDW (Red Cell Distribution Width)	31
5.2.15	Boxplot of Thrombocyte_Count.....	32



5.2.16	Boxplot of CRP (C-reactive Protein)	32
5.2.17	Overall Insights	32
5.3	Visualizing Numerical Data Using Histograms	33
5.3.1	Age	34
5.3.2	BMI (Body Mass Index)	34
5.3.3	Height	35
5.3.4	Weight	35
5.3.5	Length_of_Stay	35
5.3.6	Alvarado_Score	35
5.3.7	Paediatric_Appendicitis_Score	36
5.3.8	Appendix_Diameter	36
5.3.9	Body_Temperature	37
5.3.10	WBC_Count (White Blood Cell Count)	37
5.3.11	Neutrophil_Percentage	38
5.3.12	RBC_Count (Red Blood Cell Count)	38
5.3.13	Hemoglobin	38
5.3.14	RDW (Red Cell Distribution Width)	38
5.3.15	Thrombocyte_Count	39
5.3.16	CRP (C-reactive Protein)	39
5.3.17	Overall Insights	39
5.4	Visualizing Categorical Data Using Bar Plots	40
5.4.1	Distribution of Sex	41
5.4.2	Distribution of Management	41
5.4.3	Distribution of Severity	42
5.4.4	Distribution of Diagnosis_Presumptive	42
5.4.5	Distribution of Diagnosis	43
5.4.6	Distribution of Appendix_on_US	43
5.4.7	Distribution of Migratory_Pain	44
5.4.8	Distribution of Lower_Right_Abd_Pain	44
5.4.9	Distribution of Contralateral_Rebound_Tenderness	45
5.4.10	Distribution of Coughing_Pain	45
5.4.11	Distribution of Nausea	46
5.4.12	Distribution of Loss_of_Appetite	46
5.4.13	Distribution of Neutrophilia	47
5.4.14	Distribution of Ketones_in_Urine	47



5.4.15	Distribution of RBC_in_Urine.....	48
5.4.16	Distribution of WBC_in_Urine.....	48
5.4.17	Distribution of Dysuria	49
5.4.18	Distribution of Stool.....	49
5.4.19	Distribution of Peritonitis	50
5.4.20	Distribution of Psoas_Sign	50
5.4.21	Distribution of Ipsilateral_Rebound_Tenderness.....	51
5.4.22	Distribution of US_Performed	51
5.4.23	Distribution of Free_Fluids	51
5.5	Analysis of Factors Associated with Appendicitis Diagnosis.....	52
5.5.1	Sex vs. Diagnosis.....	52
5.5.2	BMI vs. Diagnosis	52
5.5.3	Alvarado Score vs. Diagnosis	52
5.5.4	Migratory Pain vs. Diagnosis	53
5.5.5	Lower Right Abdomen Pain vs. Diagnosis	53
5.5.6	Nausea vs. Diagnosis	53
5.5.7	Body Temperature vs. Diagnosis	54
5.5.8	WBC Count vs. Diagnosis	54
5.5.9	CRP vs. Diagnosis	54
5.5.10	Free Fluids vs. Diagnosis.....	55
6	Feature Extraction Using Pre-trained ResNet-50	55
6.1	Load a Pre-trained ResNet-50 Model.....	55
6.2	Set the Model to Evaluation Mode	55
6.3	Remove the Final Fully Connected Layer	55
6.4	Transformation Pipeline:	56
6.5	Feature Extraction.....	57
6.5.1	Read the Image:	57
6.5.2	Convert BGR to RGB:	57
6.5.3	Apply Preprocessing (Transformations):	57
6.5.4	Extract Features using ResNet-50:	57
7	Preprocessing:	58
7.1	Parsing Us_Number	58
7.1.1	Uses Regular Expressions (re.match)	58
7.1.2	Extracts and Converts the Number.....	58
	Purpose:	58



7.2	Processing and Storing Image Features.....	58
7.2.1	Initialize Storage:.....	59
7.2.2	Process Images:.....	59
7.2.3	Create a DataFrame:.....	59
7.2.4	Result:	59
	Purpose:	60
	• Extracts image features using a pre-trained model.....	60
	• Organizes these features dynamically into a structured format for further analysis or machine learning tasks.60	
7.3	Merging and Saving Processed Data.....	60
7.3.1	Merge DataFrames:	60
7.3.2	Save the Combined Data:	60
7.3.3	Confirmation Message:	60
7.3.4	Result:	60
7.4	Cleaning Data by Dropping Columns with Excessive Null Values:.....	60
7.4.1	Define a Threshold:	61
7.4.2	Display DataFrame Shapes:.....	61
	Purpose:	61
	Modeling	61
8	Random Forest.....	61
8.1	Preprocessing and Selecting Data for Random Forest.....	61
8.1.1	Define Features and Target:	61
8.1.2	Preprocessing Pipeline:	62
8.1.3	Apply Preprocessing:.....	62
8.1.4	Select a Subset of Data:	62
8.1.5	Output:	62
	Purpose:	62
8.2	Training and saving a Random Forest model.....	62
8.2.1	Display Feature Names	62
8.2.2	Split Data.....	62
8.2.3	Encode Target Labels	62
8.2.4	Train a Random Forest Model.....	62
8.2.5	Predict and Evaluate.....	63
8.2.6	Save the Model and Preprocessor	63
	Purpose:	63



8.3	Output.....	63
8.4	Overall Model Performance	64
8.5	Visualizing Model Performance.....	64
8.6	Confusion Matrix Visualization.....	64
8.6.1	Decision Tree Visualization	64
9	Logistic regression.....	65
9.1	Encoding Categorical Features for Model Training	65
9.1.1	1. Define Features	65
9.1.2	Create a Training Dataset	65
9.1.3	Apply Label Encoding	65
9.2	Training a Logistic Regression Model for Classification	66
9.2.1	Define Features (X) and Target (y)	66
9.2.2	Encode Categorical Features and Target	66
9.2.3	Split Data into Training and Testing Sets	66
9.2.4	Standardize the Features.....	66
9.2.5	Train a Logistic Regression Model	66
9.2.6	Predict and Evaluate.....	66
9.2.7	Save the Model.....	67
9.2.8	Output	67
10	SVC	68
10.1	Training and Evaluating an SVM Model for Classification.....	68
10.2	Define Features (X_tabular) and Target (y)	68
10.2.1	Split Data into Training and Testing Sets	68
10.2.2	Train an SVM Model	68
10.2.3	Predict and Evaluate.....	68
	Purpose.....	68
10.2.4	Output:	68
10.3	Model Performance & Evaluation	69
10.3.1	Comparison of Model Performance.....	69
10.3.2	Key Insights:.....	69
11	Deployment Appendicitis Prediction Web Application	69
11.1	Initialization and Model Loading.....	69
11.2	Image Feature Extraction Using ResNet-50	70
11.3	Web Interface and User Input Handling.....	70
11.4	Data Preprocessing and Feature Engineering	70



11.5	Model Prediction Process	70
11.6	Error Handling and Deployment	70
11.7	Conclusion	70
12	User Interface	71
12.1	Random Forest	71
12.2	SVC	72
12.3	Logistic Regression	73
13	Conclusion	74

1 Introduction

1.1 BACKGROUND INFORMATION

Appendicitis is one of the most common causes of pediatric abdominal surgeries, necessitating accurate and timely diagnosis to prevent complications and reduce unnecessary interventions. With recent advancements in machine learning, data-driven decision support systems have the potential to assist clinicians in diagnosing and managing suspected appendicitis cases more effectively. Traditionally, decision support systems have focused on clinical, laboratory, scoring, and computed tomography (CT) data, often overlooking the potential of abdominal ultrasound (US) imaging, a noninvasive and widely available diagnostic tool.

1.2 SIGNIFICANCE AND MOTIVATION

The study's goal is to leverage a multimodal dataset to develop interpretable machine learning models capable of predicting appendicitis-related outcomes with high accuracy. A notable methodological contribution of the study is the generalization of concept bottleneck models, which are designed to handle prediction problems involving multiple views and incomplete concept sets. These models enable clinicians to interpret and interact with high-level medical concepts without the need for extensive manual image annotation, thereby improving usability and adoption in real-world clinical settings.

By harnessing the power of machine learning, this study aims to enhance the diagnostic workflow for pediatric appendicitis, ultimately leading to improved patient outcomes and optimized resource utilization within healthcare institutions.

1.3 DATASET OVERVIEW

This report presents an analysis of a dataset acquired from a retrospective study conducted at Children's Hospital St. Hedwig in Regensburg, Germany, between 2016 and 2021. The dataset includes data from a cohort of 782 pediatric patients admitted with abdominal pain suspected of appendicitis. For each patient, the dataset provides tabular data comprising laboratory results, physical examination findings, scoring results (such as Alvarado and pediatric appendicitis scores), and ultrasonographic findings manually extracted by experts. Additionally, ultrasound images depicting various anatomical regions, including the appendix, intestines, lymph nodes, and reproductive organs, were collected with varying numbers of views per patient (ranging from 1 to 15).

The dataset incorporates three key target variables:

- **Diagnosis:** Appendicitis vs. no appendicitis
- **Management:** Surgical vs. conservative treatment
- **Severity:** Complicated vs. uncomplicated appendicitis or no appendicitis

The remainder of this report will provide an in-depth analysis of the dataset, methodologies employed, and the performance of various predictive models developed for the classification tasks associated with diagnosis, management, and severity of suspected appendicitis.

1.4 Tools and Technologies

To deal with the aims and objectives of the project we chose to use the following tools and technologies for our project. They are as follows:

1.4.1 HARDWARE

The development process was done by the following laptops as shown:

Description	Configuration
Laptops	HP Pavilion Laptop 15

1.4.2 SOFTWARE

The development process was done by the following operating systems and software as shown:

Description	Technology
Operating Systems	Windows 11
Editor	VS Code

1.4.3 DEVELOPMENT SIDE TECHNOLOGIES

The development process was done by the following technologies as shown:

Description	Technology
Language	Python v3.12.1
Collaboration	GitHub
Summarizer Engine	VS Code v1.86.1

Since the inception of this project, we have taken everything under consideration with respect to research. From gathering the prerequisites of the project to deploying it, the complete guidance for each module of the project was taken under study.

1.5 Architecture

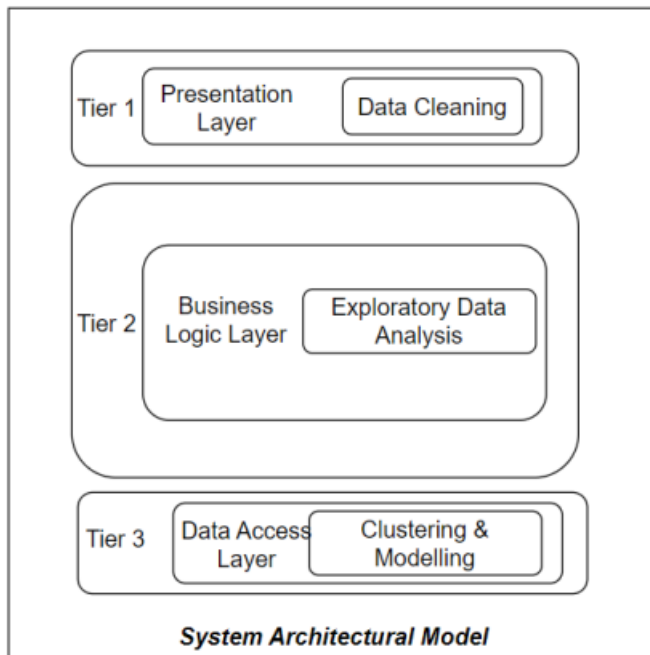
1.5.1 Introduction

This section provides an outline of the project, its integral parts and major data objects involved and their relationship that flows in the system. It gives us basic understanding about system architecture, how the data objects interact with the system and the overall workflow. The overall running process and the data and interface description are briefly described in this section below.

1.5.2 System architecture

To build a scalable, efficient and maintainable software system, layered architecture was built for separation of concern, scalability and maintenance. Each layer is independent of each other as they are configured separately for each process. If need arises, these layers can be replaced by various implementations.

1.5.3 Architecture model



1.6 Exploratory data analysis

Exploratory Data Analysis (EDA) is a crucial preliminary step in the data analysis process, aimed at gaining insights, understanding patterns, and uncovering relationships within the dataset. In a report, an overview of EDA typically covers the following key aspects:

1. Introduction to EDA:

Provide a brief introduction to EDA, explaining its purpose and importance in the data analysis process. Highlight how EDA serves as a foundational step to understand the dataset before applying more advanced analytical techniques.

2. Dataset Description:

Introduce the dataset under analysis, including its source, scope, and variables/features included. Provide a high-level overview of the data structure, such as the number of observations (rows) and variables (columns), as well as any metadata available.

3. Summary Statistics:

Present summary statistics of the dataset to describe its central tendency, variability, and distribution. This may include measures such as mean, median, standard deviation, minimum, maximum, and quartiles for numerical variables, as well as frequency tables for categorical variables.

4. Data Visualization:

Utilize visualizations such as histograms, box plots, scatter plots, and bar charts to explore the distribution and relationships between variables. Visualizations help identify patterns, outliers, and potential relationships that may not be evident from summary statistics alone.

5. Univariate Analysis: Conduct univariate analysis to explore the distribution of individual variables in isolation. Examine the distribution of numerical variables for skewness, kurtosis, and potential outliers, and analyse the frequency distribution of categorical variables.

6. Bivariate Analysis:

Perform bivariate analysis to explore relationships between pairs of variables. Investigate correlations between numerical variables using scatter plots or correlation matrices and examine relationships between categorical and numerical variables using box plots or bar charts.

7. Multivariate Analysis:

Explore interactions between multiple variables through multivariate analysis. Utilize techniques such as heatmaps, pair plots, and multidimensional scaling to visualize relationships between multiple variables simultaneously.

8. Missing Data and Outlier Detection:

Identify and address missing data and outliers in the dataset. Describe methods for handling missing values, such as imputation or deletion, and techniques for detecting and treating outliers, such as winsorization or robust statistical methods.

9. Conclusion:

Summarize the key findings and insights gained from the EDA process. Highlight any notable patterns, trends, or relationships discovered, and discuss implications for further analysis or decision-making.

2 Comparison of ML Models for Appendicitis Diagnosis

In this study, multiple machine learning models were explored to classify appendicitis cases using a combination of tabular and image data. The models implemented include **Random Forest (RF)**, **Support Vector Classification (SVC)**, and **Logistic Regression (LR)**. Each model offers unique strengths and limitations, which are analyzed below.

2.1 Random Forest (RF)

2.1.1 Overview

Random Forest is an **ensemble learning method** that operates by building multiple decision trees and combining their outputs to enhance accuracy and robustness. It is widely used for both classification and regression tasks.

2.1.2 Key Concepts

1. **Decision Trees** – The model constructs multiple decision trees using different subsets of the training data (bootstrapping).
2. **Ensemble Learning** – Predictions from all decision trees are combined using majority voting for classification, reducing overfitting and improving generalization.
3. **Feature Importance** – RF evaluates feature importance, helping identify key variables influencing appendicitis diagnosis.

2.1.3 Algorithm Steps

- Randomly select a subset of data samples with replacement (**bootstrap sampling**).

- Construct multiple decision trees, each trained on different feature subsets.
- Aggregate predictions using majority voting for classification.

2.1.4 Mathematical Representation

Given a set of trained decision trees $T_1, T_2, \dots, T_n$, the final prediction $\hat{y}$ is determined by majority voting:

$$\hat{y} = \text{mode}(T_1(x), T_2(x), \dots, T_n(x))$$

where x represents an input instance.

2.1.5 Advantages of Random Forest

- ✓ Handles high-dimensional data effectively.
- ✓ Reduces overfitting compared to a single decision tree.
- ✓ Provides reliable predictions with high accuracy.

2.1.6 Limitations

- ✗ Computationally expensive for large datasets.
- ✗ Requires tuning of hyperparameters (e.g., number of trees, depth).

2.2 Support Vector Classification (SVC)

2.2.1 Overview

Support Vector Classification (SVC) is a **supervised learning algorithm** that aims to find the optimal **hyperplane** that maximizes the margin between different classes in a high-dimensional space. It is particularly useful for binary classification problems.

2.2.2 Key Concepts

- **Hyperplane** – The decision boundary that best separates two classes.
- **Support Vectors** – Data points closest to the decision boundary that influence the hyperplane's position.
- **Kernel Trick** – SVC can map data to higher dimensions using kernel functions to handle non-linearly separable cases.

2.2.3 Algorithm Steps

- Identify the optimal hyperplane that maximizes the margin between classes.
- Use kernel functions (linear, polynomial, RBF) to transform data if needed.
- Solve an optimization problem to minimize classification error.

2.2.4 Mathematical Representation

The decision function in SVC is given by:

$$f(x) = \sum_{i=1}^n \alpha_i y_i K(x_i, x) + b$$

where:

- α_i are the Lagrange multipliers.
- y_i are the class labels.
- $K(x_i, x)$ is the kernel function.
- b is the bias term.

2.2.5 Advantages of SVC

- ✓ Works well in high-dimensional spaces.
- ✓ Effective for small to medium-sized datasets.
- ✓ Robust to overfitting, especially in complex classification tasks.

2.2.6 Limitations

- ✗ Computationally intensive for large datasets.
- ✗ Requires careful kernel selection and tuning.

2.3 Logistic Regression (LR)

2.3.1 Overview

Logistic Regression is a **probabilistic classification model** used for binary classification. Unlike linear regression, LR uses the **sigmoid function** to output probabilities between 0 and 1, making it suitable for decision-making tasks.

2.3.2 Key Concepts

- **Sigmoid Function** – Maps predictions to a probability range $[0,1]$.
- **Decision Boundary** – The threshold (typically 0.5) that determines class assignment.
- **Maximum Likelihood Estimation (MLE)** – Used to estimate model coefficients.

2.3.3 Algorithm Steps

- Compute the weighted sum of features using learned coefficients.
- Apply the sigmoid activation function.
- Classify based on probability threshold (e.g., 0.5).

2.3.4 Mathematical Representation

The probability of the outcome is computed as:

$$P(Y=1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n)}} \quad P(Y=1 | X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n)}}$$

where:

- β_0 is the intercept.
- $\beta_1, \beta_2, \dots, \beta_n$ are feature coefficients.
- $X_1, X_2, \dots, X_n$ are input features.

2.3.5 Advantages of Logistic Regression

- ✓ Simple and easy to interpret.
- ✓ Provides class probabilities.
- ✓ Works well when data is linearly separable.

2.3.6 Limitations

- ✗ Struggles with complex non-linear relationships.
- ✗ Assumes independence of features.

3 Libraries

```
import os
import re
import cv2
import torch
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from torchvision import models, transforms
from torch.nn import functional as F
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncoder, LabelEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.tree import plot_tree
```

This script imports a variety of libraries and modules necessary for machine learning, computer vision, and data analysis:

3.1 Deep Learning & Computer Vision:

- torch, torchvision: For working with PyTorch deep learning models.
- cv2: For image processing (OpenCV).

3.2 Data Processing & Transformation:

- numpy, pandas: For handling and processing numerical and tabular data.
- sklearn.preprocessing (StandardScaler, OrdinalEncoder, LabelEncoder, etc.): For scaling, encoding, and transforming datasets.
- SimpleImputer: Handles missing values in data.

3.3 Model Training & Evaluation:

- sklearn.model_selection (train_test_split): Splits datasets into training and testing sets.
- sklearn.ensemble (RandomForestClassifier): A machine learning model for classification.
- sklearn.linear_model (LogisticRegression): A statistical model for binary/multiclass classification.
- sklearn.metrics (classification_report, confusion_matrix, accuracy_score): Evaluates model performance.

3.4 Visualization:

- matplotlib, seaborn: For creating static plots and visualizations.
- plotly.express, plotly.graph_objects: For interactive visualizations.
- sklearn.tree.plot_tree: Visualizes decision trees.

This setup provides the necessary tools for data preprocessing, model training, evaluation, and visualization in a machine learning project.

4 Functional and Data Description

4.1 Introduction

This section describes the main modules of this application, sub-modules and their components that are implemented in order to achieve required results.

4.2 Data loading

We dynamically assign file paths based on user input and subsequently load tabular data for analysis. It utilizes a while loop to prompt the user to enter their name and assigns the corresponding file paths for data processing based on the input. The script supports three predefined users—Hassan, Muneeb, and Sameer—each having unique file locations for tabular data, image directory, and output storage.

Upon entering a valid name, the script assigns appropriate paths for:

- **Tabular Data Path:** Specifies the location of the Excel file containing structured patient data.
- **Image Directory:** Points to the folder containing ultrasound images.
- **Output File Path:** Defines where the processed data will be stored after analysis.

If an invalid name is entered, the program prints an error message and prompts the user to re-enter their name until a correct match is found. Once a valid name is provided, the code proceeds to load the Excel file using the pandas library, displaying the first few rows to confirm successful data loading. After that we just printed few rows from our dataset

Adding paths

```
while True:
    c = input('Enter your name: ').lower()
    if c == 'hassan':
        tabular_data_path = 'D:/Masters/Semester2/Data mining/Project/DataMining/Dataset/app_data.xlsx'
        image_dir = 'D:/Masters/Semester2/Data mining/Project/DataMining/Dataset/Images'
        to_save = 'D:/Masters/Semester2/Data mining/Project/DataMining/Dataset/merged_data_with_image_features.csv'
        break
    elif c == 'muneeb':
        tabular_data_path = 'C:/Users/Syed Muhammad Muneeb/Downloads/DataMiningDS/app_data.xlsx'
        image_dir = 'C:/Users/Syed Muhammad Muneeb/Downloads/DataMiningDS'
        to_save = 'C:/Users/Syed Muhammad Muneeb/Downloads/DataMiningDS/merged_data_with_image_features.csv'
        break
    elif c == 'sameer':
        tabular_data_path = 'E:/Masters Data Science/2nd Sem/Data mining/Project/Project dataset/app_data.xlsx'
        image_dir = 'E:/Masters Data Science/2nd Sem/Data mining/Project/Project dataset/US_Pictures/US_Pictures/'
        to_save = 'E:/Masters Data Science/2nd Sem/Data mining/Project/Project dataset/merged_data_with_image_features.csv'
    else:
        print('Ops! You made a boo boo please check your name again')
        continue
```

Enter your name: hassan

```
print(tabular_data_path)
```

D:/Masters/Semester2/Data mining/Project/DataMining/Dataset/app_data.xlsx

```
#Load your tabular data
df = pd.read_excel(tabular_data_path)

# Display the first few rows of the tabular data
print(df.head())
```

	Age	BMI	Sex	Height	Weight	Length_of_Stay	Management	\
0	12.68	16.9	female	148.0	37.0	3.0	conservative	
1	14.10	31.9	male	147.0	69.5	2.0	conservative	
2	14.14	23.3	female	163.0	62.0	4.0	conservative	
3	16.37	20.6	female	165.0	56.0	3.0	conservative	
4	11.08	16.9	female	163.0	45.0	3.0	conservative	

4.3 Comprehensive Dataset Summary for Initial Exploration and Preprocessing

The implemented approach generates a comprehensive summary of the dataset to facilitate initial data exploration and preprocessing. We created a dictionary, summary, that includes key dataset details such as shape (number of rows and columns), column data types, missing values, and sample data.

We then categorized columns into numerical and categorical types using the `select_dtypes()` function, followed by computing descriptive statistics for each type. Numerical columns were summarized with metrics like mean and standard deviation, while categorical columns were analyzed for unique values and frequencies.

Finally, the summary dictionary was updated with the extracted insights, providing a structured overview of the dataset. This approach helped in identifying data quality issues, understanding variable distributions, and preparing the data for further processing and analysis.

```
# Summary of the dataset
summary = {
    "Shape": df.shape,
    "Info": df.info(),
    "Missing Values": df.isnull().sum(),
    "Sample Data": df.head(),
}

# Separate numerical and categorical columns
numerical_columns = df.select_dtypes(include=['number']).columns
categorical_columns = df.select_dtypes(include=['object', 'category']).columns

# Summary of numerical and categorical columns
numerical_summary = df[numerical_columns].describe()
categorical_summary = df[categorical_columns].describe()

summary.update({
    "Numerical Columns": numerical_columns.tolist(),
    "Categorical Columns": categorical_columns.tolist(),
    "Numerical Summary": numerical_summary,
    "Categorical Summary": categorical_summary,
})

summary
```

'Numerical Summary':						
	Age	BMI	Height	Weight	Length_of_Stay	\
count	781.000000	755.000000	756.000000	779.000000	778.000000	
mean	11.346483	18.906916	148.017460	43.172542	4.284062	
std	3.529979	4.385252	19.732016	17.390984	2.574057	
min	0.000000	7.827983	53.000000	3.960000	1.000000	
25%	9.200000	15.725294	137.000000	29.500000	3.000000	
50%	11.438741	18.062284	149.650000	41.400000	3.000000	
75%	14.099932	21.179011	163.000000	54.000000	5.000000	
max	18.360000	38.156221	192.000000	103.000000	28.000000	

Alvarado_Score Paedriatic_Appendicitis_Score Appendix_Diameter \			
	Alvarado_Score	Paedriatic_Appendicitis_Score	Appendix_Diameter
count	730.000000	730.000000	498.000000
mean	5.921918	5.253425	7.762651
std	2.155972	1.958456	2.536671
min	0.000000	0.000000	2.700000
25%	4.000000	4.000000	6.000000
50%	6.000000	5.000000	7.500000
75%	8.000000	7.000000	9.100000
max	10.000000	10.000000	17.000000

4.4 Assessing Missing Values in the Dataset

The implemented approach calculates the total and percentage of missing values in the dataset to facilitate data quality assessment. We computed the total null values for each column using the `isnull().sum()` function and calculated the corresponding percentage by dividing the total null values by the dataset length and multiplying by 100.

A summary table was then created using the Pandas `DataFrame` function, containing columns for the total number of null values and their corresponding percentages. The table was sorted in descending order based on the total null values to prioritize columns with the highest missing data, and the index was reset for better readability.

This approach helps in identifying missing data patterns, assessing data completeness, and making informed decisions regarding data imputation or removal strategies.

```
# Calculate the total and percentage of null values in the dfset
null_values = df.isnull().sum()
null_percentage = (null_values / len(df)) * 100

# Create a summary table for columns with null values
null_summary = pd.DataFrame({
    "Column": df.columns,
    "Total_Null_Values": null_values,
    "Percentage_Null_Values": null_percentage
}).sort_values(by="Total_Null_Values", ascending=False).reset_index(drop=True)

null_summary
```

	Column	Total_Null_Values	Percentage_Null_Values
0	Abscess_Location	769	98.337596
1	Gynecological_Findings	756	96.675192
2	Conglomerate_of_Bowel_Loops	739	94.501279
3	Segmented_Neutrophils	728	93.094629
4	Ileus	722	92.327366
5	Perfusion	719	91.943734
6	Enteritis	716	91.560102
7	Appendicolith	713	91.176471
8	Coprostasis	711	90.920716
9	Perforation	701	89.641944
10	Appendicular_Abscess	697	89.130435
11	Bowel_Wall_Thickening	683	87.340153
12	Lymph_Nodes_Location	661	84.526854
13	Target_Sign	644	82.352941
14	Meteorism	642	82.097187
15	Pathological_Lymph_Nodes	579	74.040921
16	Appendix_Wall_Layers	564	72.122762
17	Surrounding_Tissue_Reaction	530	67.774936
18	Appendix_Diameter	284	36.317136
19	RBC_in_Urine	206	26.342711
20	Ketones_in_Urine	200	25.575448
21	WBC_in_Urine	199	25.447570
22	Ipsilateral_Rebound_Tenderness	163	20.843990
23	Neutrophil_Percentage	103	13.171355
24	Free_Fluids	63	8.056266
25	Alvarado_Score	52	6.649616
26	Paedriatic_Appendicitis_Score	52	6.649616

27	Neutrophilia	50	6.393862
28	Psoas_Sign	37	4.731458
29	Dysuria	29	3.708440
30	BMI	27	3.452685
31	Height	26	3.324808
32	RDW	26	3.324808
33	US_Number	22	2.813299
34	Thrombocyte_Count	18	2.301790
35	Hemoglobin	18	2.301790
36	RBC_Count	18	2.301790
37	Stool	17	2.173913
38	Coughing_Pain	16	2.046036
39	Contralateral_Rebound_Tenderness	15	1.918159
40	CRP	11	1.406650
41	Loss_of_Appetite	10	1.278772
42	Peritonitis	9	1.150895
43	Migratory_Pain	9	1.150895
44	Nausea	8	1.023018
45	Lower_Right_Abd_Pain	8	1.023018
46	Body_Temperature	7	0.895141
47	WBC_Count	6	0.767263
48	Appendix_on_US	5	0.639386
49	Length_of_Stay	4	0.511509
50	US_Performed	4	0.511509
51	Weight	3	0.383632
52	Diagnosis	2	0.255754
53	Diagnosis_Presumptive	2	0.255754
54	Sex	2	0.255754
55	Severity	1	0.127877

4.5 Data Cleaning:

4.5.1 Handling Missing Data by Removing Columns with Excessive Null Values

The implemented approach focuses on handling missing data by removing columns with excessive null values. We first calculated a threshold, set at 50% of the total number of rows in the dataset, using `len(df) * 0.5`. This threshold serves as a benchmark to identify columns with a high proportion of missing values.

Columns where the number of null values exceeds this threshold were dropped using the `df.loc[:, df.isnull().sum() <= threshold]` statement, ensuring only columns with an acceptable level of missing data are retained.

Finally, the shape of the original and cleaned datasets was displayed to highlight the impact of the cleaning process. This approach helps in reducing noise in the dataset, improving data quality, and ensuring that the retained features have sufficient data for meaningful analysis.

```
# Calculate the threshold for 60% null values
threshold = len(df) * 0.5
|
# Drop columns where the number of null values exceeds the threshold
df_cleaned = df.loc[:, df.isnull().sum() <= threshold]

# Display the shape of the cleaned dataframe
print("Original shape:", df.shape)
print("Cleaned shape:", df_cleaned.shape)
```

```
Original shape: (782, 58)
Cleaned shape: (782, 40)
```

The implemented approach effectively handles missing values to enhance data quality and consistency. Rows with missing values in the 'US_Number' column are removed to retain essential information. Missing numerical values are filled with the column mean, ensuring statistical integrity, while missing categorical values are replaced with the most frequent value (mode) to maintain consistency. These preprocessing steps help prepare the dataset for further analysis and machine learning applications by reducing data loss and improving completeness.

```
# Remove rows where 'US_Number' has null values
df_cleaned = df_cleaned.dropna(subset=['US_Number'])
```

```
# Fill null values in numerical columns with the mean
df_cleaned[numerical_columns] = df_cleaned[numerical_columns].fillna(df_cleaned[numerical_columns].mean())
```

Null values in numerical columns after filling:

```
# Fill null values in categorical columns with the mode
df_cleaned[categorical_columns] = df_cleaned[categorical_columns].apply(lambda col: col.fillna(col.mode()[0]))
```

Null values in categorical columns after filling:

4.6 Variable Classification

4.6.1 Numerical variables

```
numerical_columns = df_cleaned.select_dtypes(include=["number"]).columns  
print("Numerical Columns:")  
print(numerical_columns)
```

Age
BMI
Height
Weight
Length_of_Stay
Alvarado_Score
Paedriatic_Appendicitis_Score
Appendix_Diameter
Body_Temperature
WBC_Count
Neutrophil_Percentage
RBC_Count
Hemoglobin
RDW
Thrombocyte_Count
CRP
US_Number

4.6.2 CATEGORICAL VARIABLES

```
categorical_columns = df_cleaned.select_dtypes(include=["object", "category"]).columns
print("\nCategorical Columns:")
print(categorical_columns)
```

Sex

Management

Severity

Diagnosis_Presumptive

Diagnosis

Appendix_on_US

Migratory_Pain

Lower_Right_Abd_Pain

Contralateral_Rebound_Tenderness

Coughing_Pain

Nausea

Loss_of_Appetite

Neutrophilia

Ketones_in_Urine

RBC_in_Urine

WBC_in_Urine

Dysuria

Stool

Peritonitis

Psoas_Sign

Ipsilateral_Rebound_Tenderness

US_Performed

Free_Fluids

5 Exploratory Data Analysis and Visualization

In this section the exploratory analysis is used to shape the data into some meaningful form and visual representation.

5.1 Correlation Heatmap:

A **heatmap** is a data visualization technique that represents numerical values using color gradients, making it easier to identify patterns, correlations, and trends in complex datasets. Each cell in a heatmap corresponds to a data point, with its color intensity indicating the magnitude of the value.

```
# Select only numerical columns
numerical_df = df_cleaned.select_dtypes(include=['number'])
# Calculate the correlation matrix
correlation_matrix = numerical_df.corr()

# Create the heatmap
heatmap = go.Figure(
    data=[go.Heatmap(
        z=correlation_matrix.values, # Correlation values
        x=correlation_matrix.columns, # Column Labels
        y=correlation_matrix.columns, # Row Labels
        colorscale="Viridis", # Choose a color scale
        colorbar=dict(title="Correlation"), # Add a color bar
        hoverongaps=False # Display gaps in missing data
    )
])

# Update Layout for better appearance
heatmap.update_layout(
    title="Correlation Heatmap for Numerical Columns",
    xaxis=dict(title="Columns", tickangle=45), # Rotate x-axis labels
    yaxis=dict(title="Columns"),
    height=800, # Adjust height
    width=800, # Adjust width
    margin=dict(l=50, r=50, t=50, b=100) # Add margin for readability
)

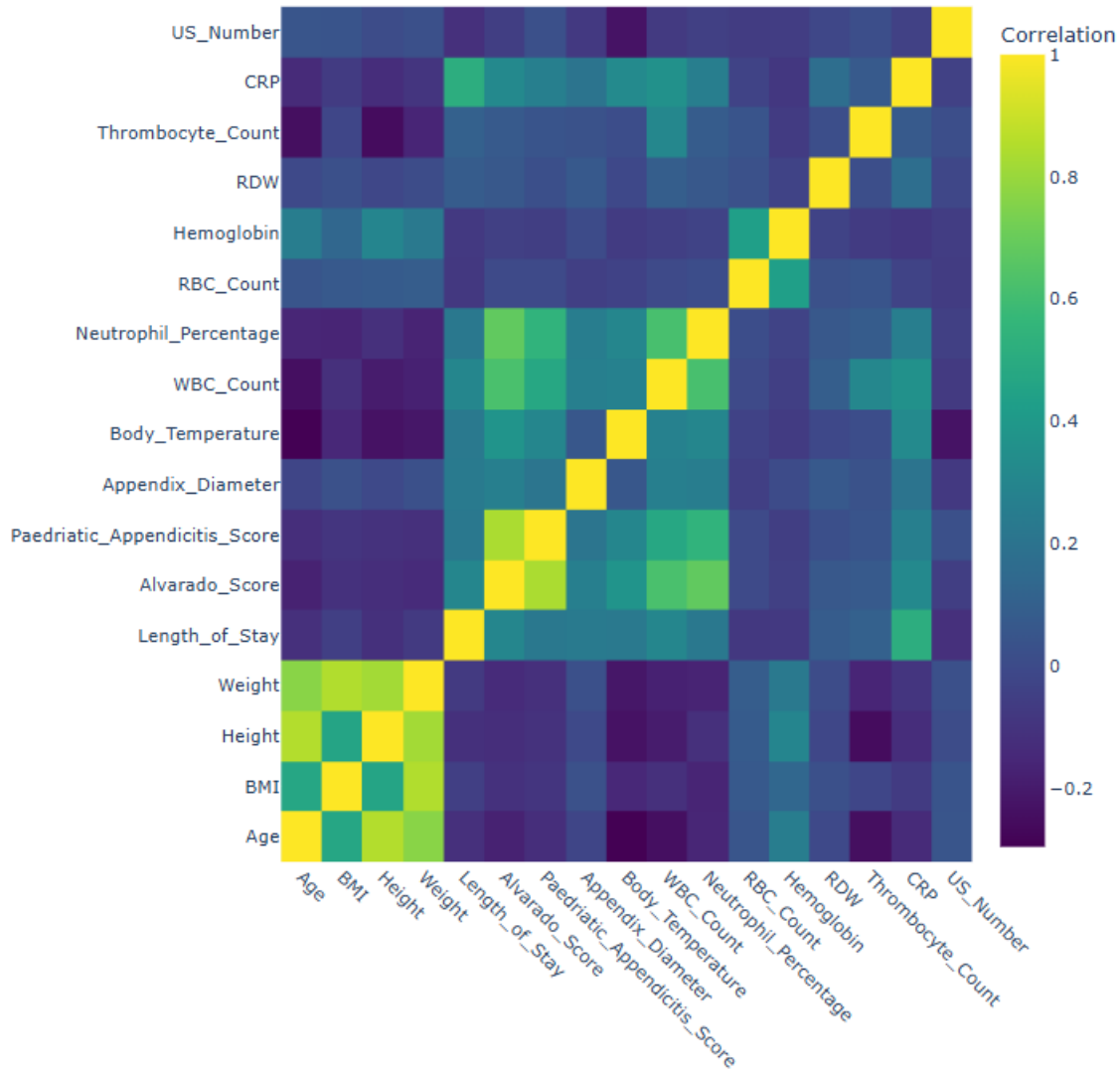
# Display the heatmap
heatmap.show()
```

5.1.1 Observation:

The correlation heatmap visually represents the relationships between numerical variables in the dataset. The diagonal values show perfect self-correlation (yellow), while other cells indicate the strength and direction of relationships, ranging from strong positive (yellow-green) to weak/no correlation (blue-purple).

Key observations include strong correlations among Height, Weight, and BMI, while variables like CRP and US_Number show weaker correlations with others. Highly correlated features may introduce redundancy and should be considered for feature selection, whereas weak correlations suggest unique contributions.

This analysis helps in identifying feature dependencies, guiding data preprocessing, and improving model performance through informed feature selection.



5.2 Box Plots:

The box plots provide a visual representation of the distribution, central tendency, and outliers present in key numerical variables related to pediatric appendicitis patients. The analysis of each variable helps in understanding its spread and identifying potential data quality issues.

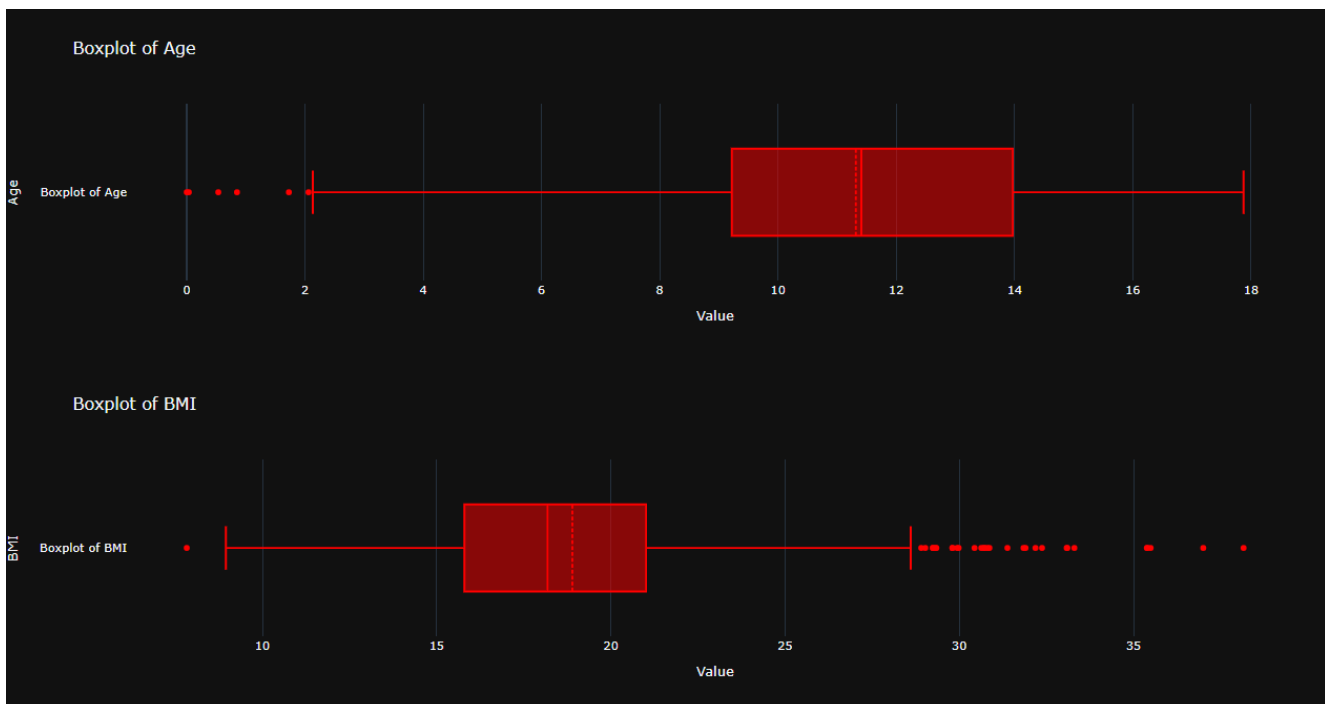
```
# Assuming 'df' is your DataFrame
numerical_columns = df.select_dtypes(include=["number"]).columns

for col in numerical_columns:
    # Create the horizontal boxplot
    fig = go.Figure()

    # Add the boxplot trace
    fig.add_trace(go.Box(
        x=df[col], # Data for the boxplot (x-axis for horizontal orientation)
        boxmean=True, # Show the mean
        name=f"Boxplot of {col}",
        marker=dict(color='red'),
        orientation='h' # Horizontal boxplot
    ))

    # Update Layout
    fig.update_layout(
        title=f"Boxplot of {col}",
        xaxis_title="Value", # x-axis title for the horizontal boxplot
        yaxis_title=col, # y-axis title for the column
        template="plotly_dark", # Optional: for a dark theme
    )

    # Show the plot
    fig.show()
```

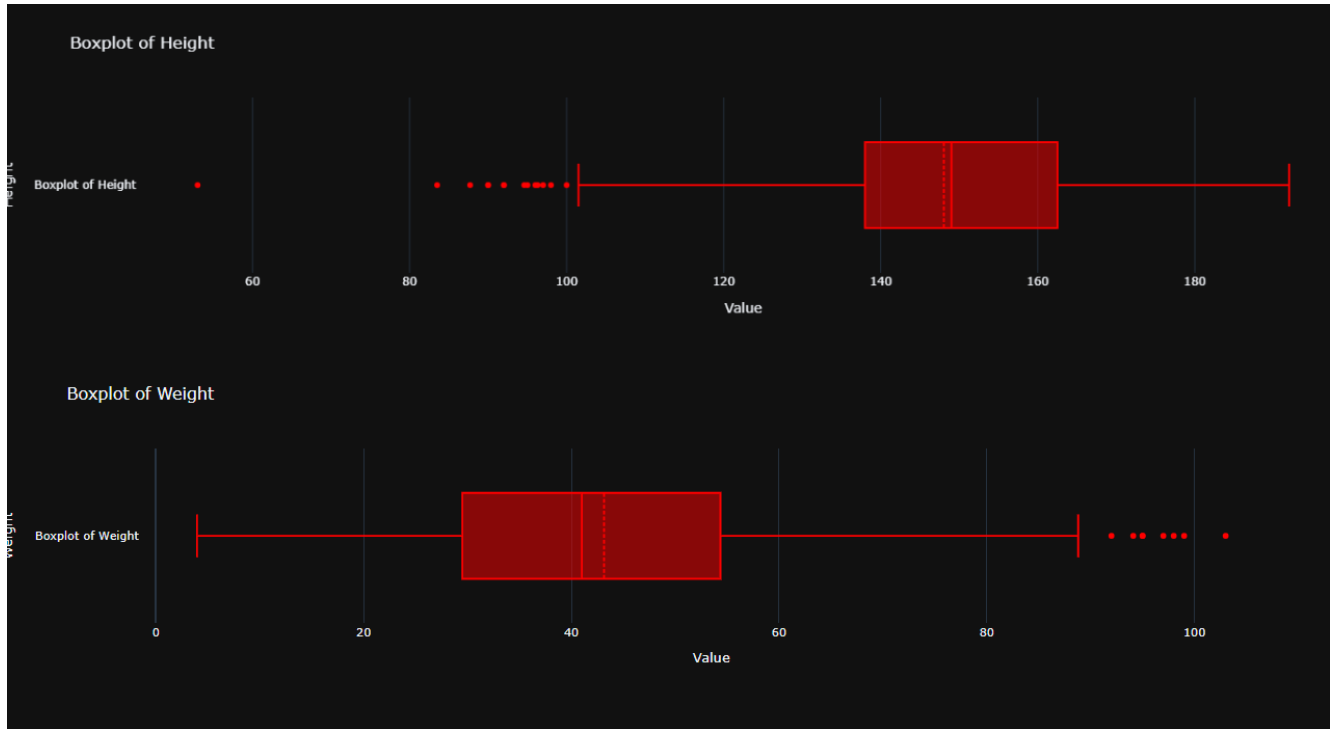


5.2.1 Boxplot of Age

- The age values primarily range between 9 and 14 years, with a median around 11-12 years.
- Outliers below 2 years indicate very young patients and may require further validation.
- The distribution suggests that most patients belong to the pre-adolescent age group.

5.2.2 Boxplot of BMI (Body Mass Index)

- BMI values mostly fall within the range of 15 to 25, with a median around 20.
- Several outliers above 30 suggest the presence of overweight or obese patients.
- The distribution indicates a right skew, with a few patients having higher BMI values.

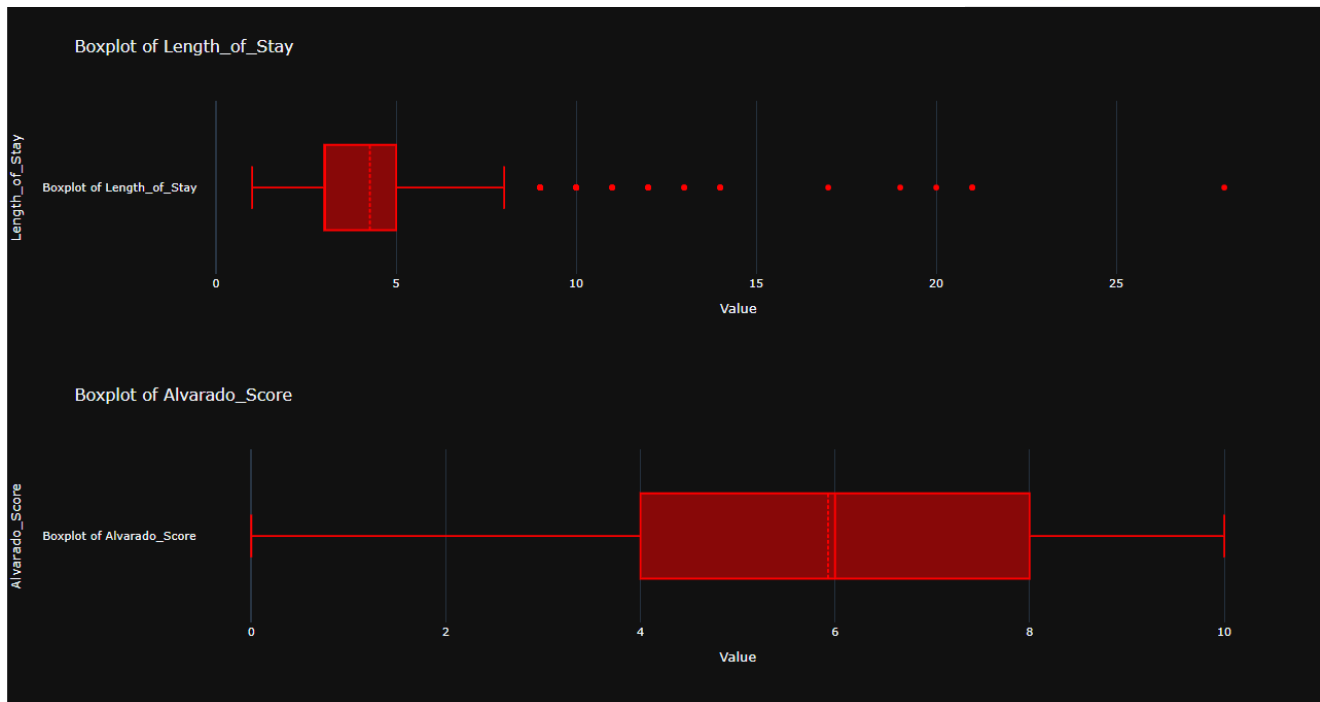


5.2.3 Boxplot of Height

- The interquartile range (IQR) of height lies between 140 and 170 cm, with a median around 160 cm.
- Several low outliers below 100 cm indicate younger or shorter patients.
- The distribution is right-skewed, showing that most patients have average height, with a few significantly shorter cases.

5.2.4 Boxplot of Weight

- The majority of weight values fall between 30 and 60 kg, with a median around 45 kg.
- Outliers beyond 80 kg may indicate overweight individuals.
- A left-skewed distribution suggests that a majority of the patients have lower weights.

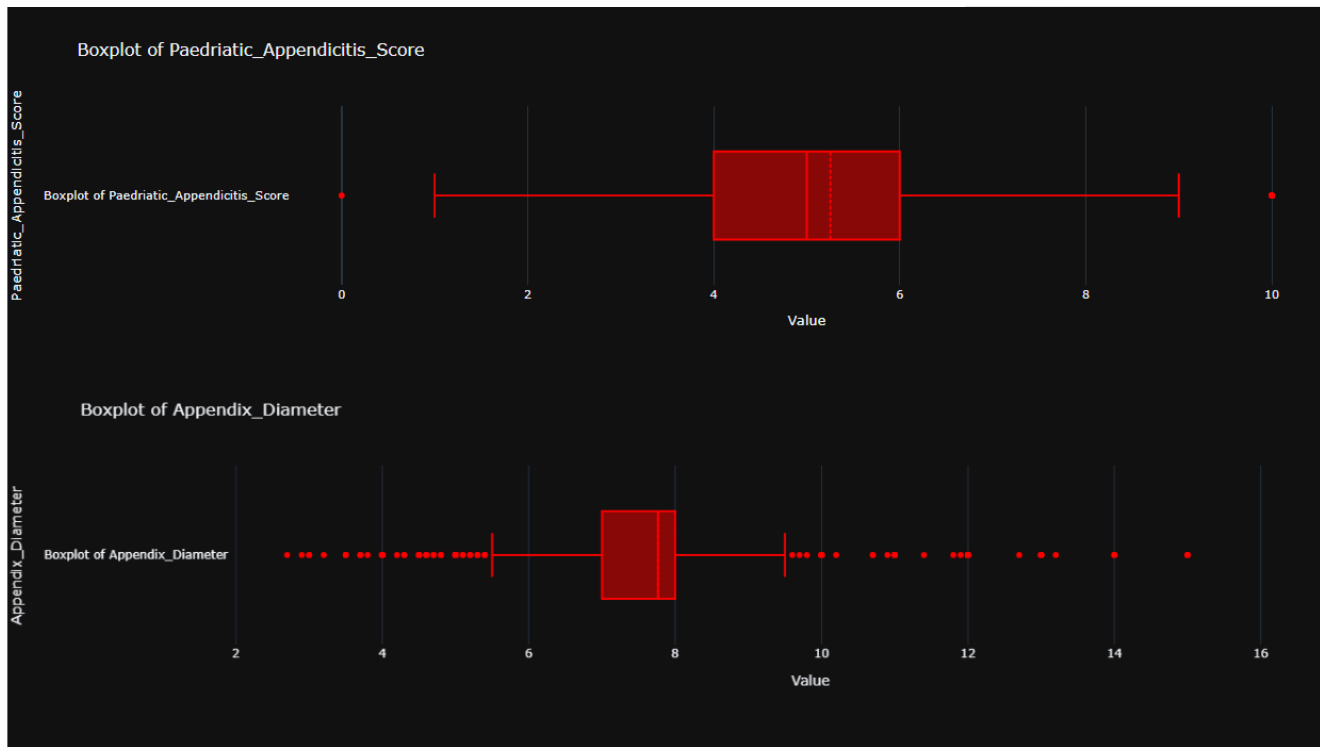


5.2.5 Boxplot of Length_of_Stay

- The hospital stay duration mainly ranges between 2 and 6 days, with a median around 4 days.
- Several outliers extend beyond 10 days, indicating prolonged hospital stays.
- The distribution suggests that most patients are discharged within a week.

5.2.6 Boxplot of Alvarado_Score

- The Alvarado score ranges from 0 to 10, with most values between 4 and 8, indicating moderate risk levels.
- No significant outliers are observed, showing a well-distributed dataset.
- The median score around 6 aligns with the typical diagnostic criteria for appendicitis.

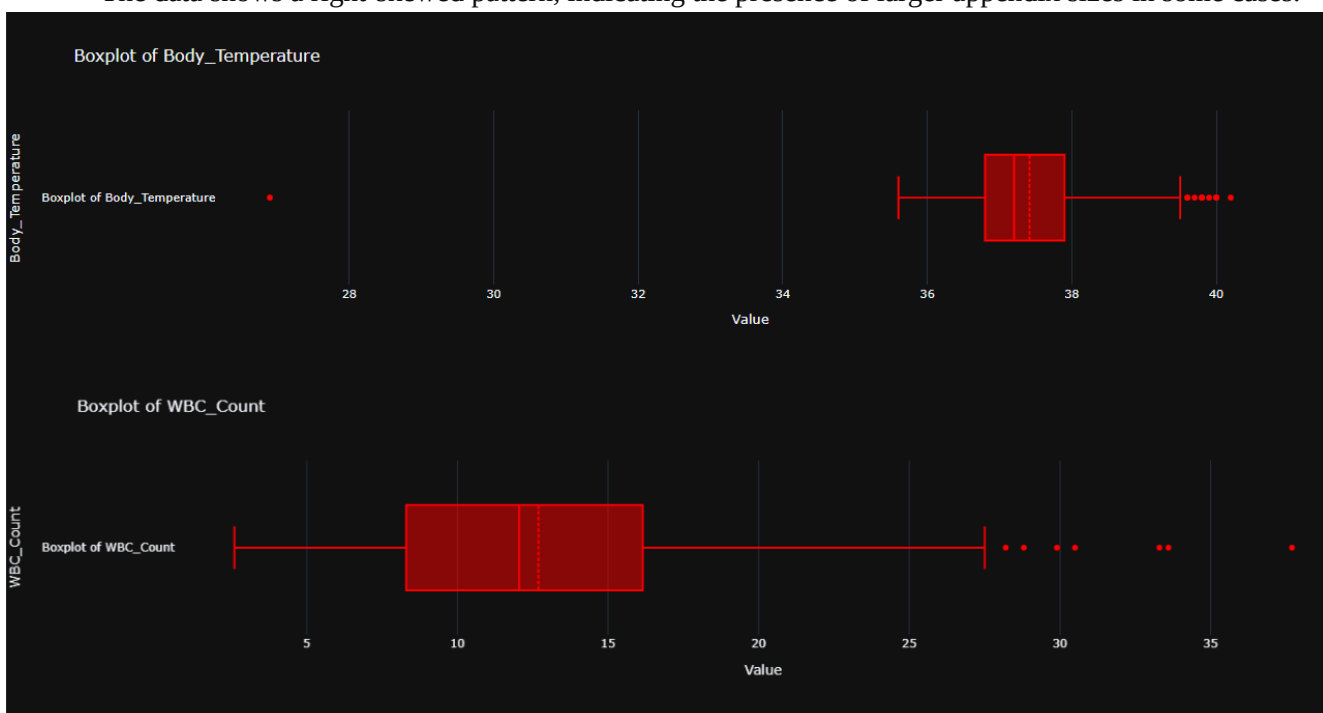


5.2.7 Boxplot of Paediatric_Appendicitis_Score

- The scores mostly fall within the range of 4 to 6, with a median around 5.
- Some outliers at 0 and 10 may represent extreme cases of misdiagnosis or severe appendicitis.
- The distribution is relatively balanced without strong skewness.

5.2.8 Boxplot of Appendix_Diameter

- Appendix diameter ranges between 6 and 9 mm, with a median around 8 mm.
- Numerous outliers above 10 mm suggest potential cases of appendicitis.
- The data shows a right-skewed pattern, indicating the presence of larger appendix sizes in some cases.

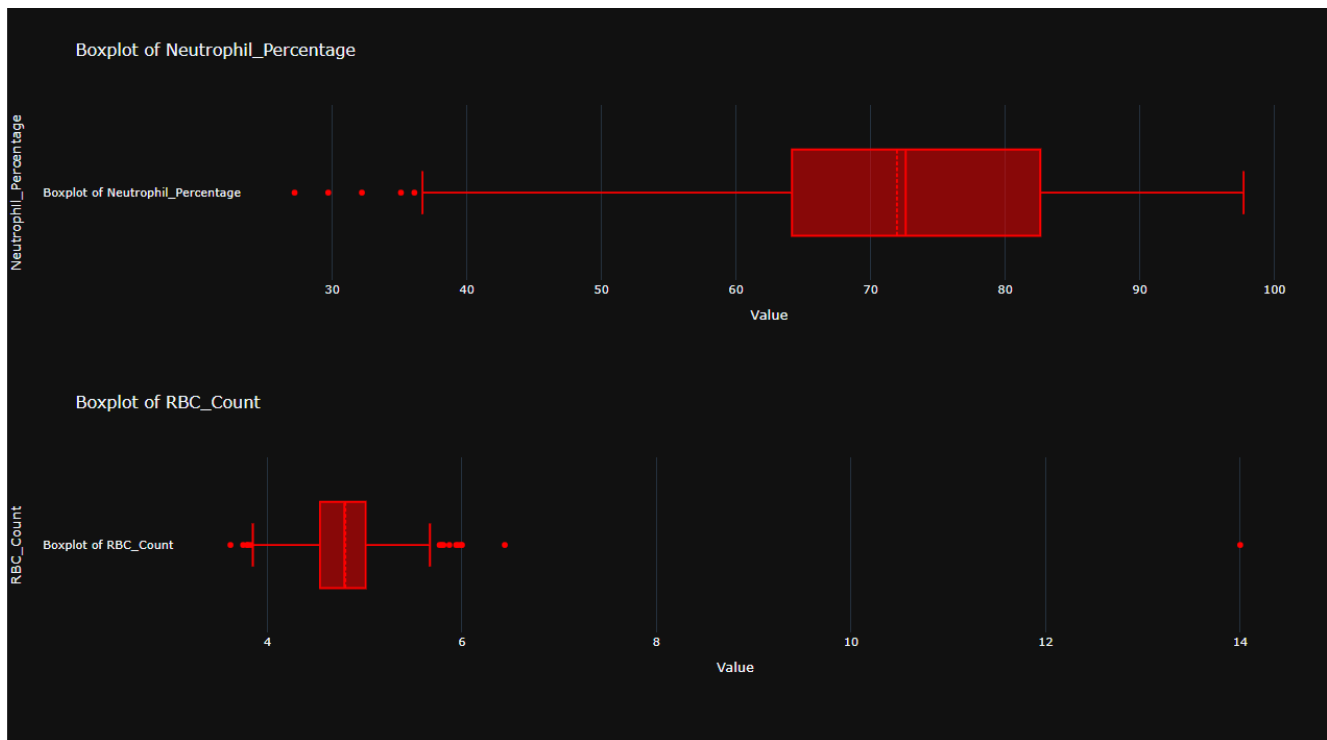


5.2.9 Boxplot of Body_Temperature

- The typical range is between 36.5°C and 38°C, with a median around 37°C.
- A significant outlier below 30°C suggests potential data entry errors.
- Fever cases are indicated by values above 40°C, which may correlate with appendicitis severity.

5.2.10 Boxplot of WBC_Count (White Blood Cell Count)

- WBC counts are primarily within 7 to 15, with a median around 10.
- Outliers above 25 may indicate severe infections or inflammation.
- A slightly right-skewed distribution suggests that higher values are more common in appendicitis cases.

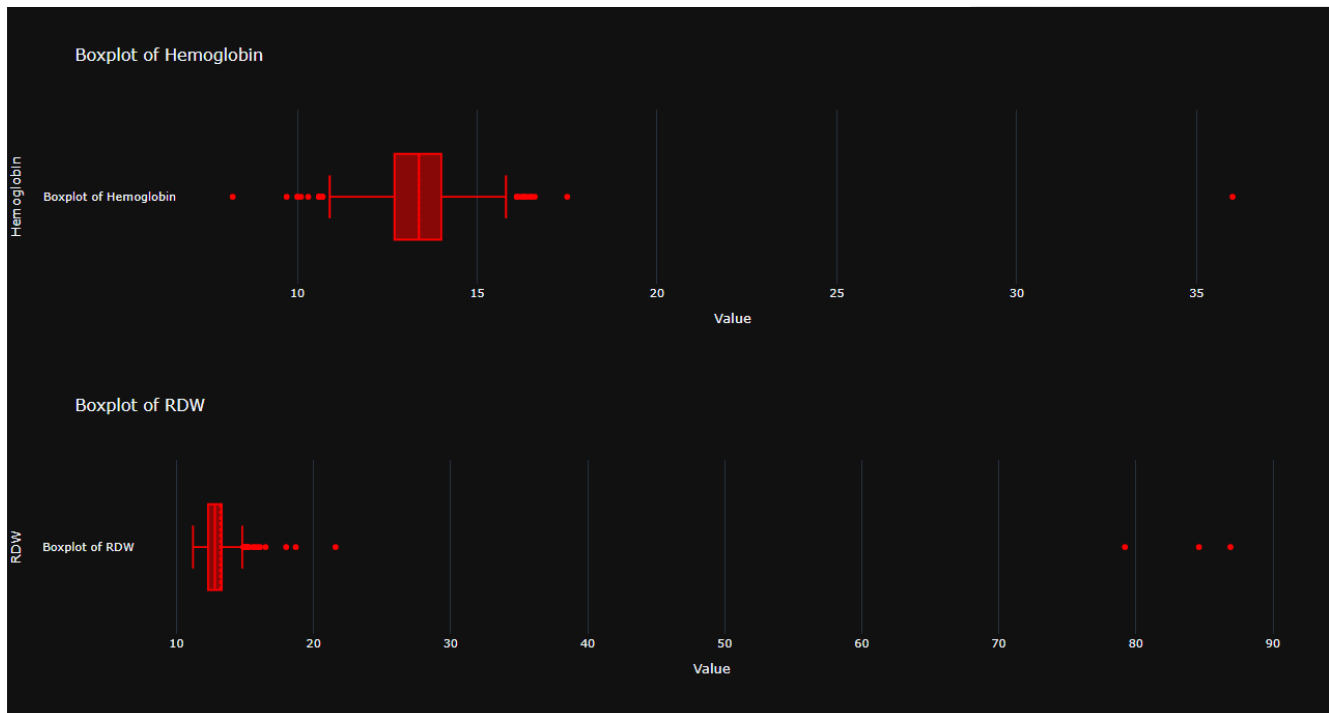


5.2.11 Boxplot of Neutrophil_Percentage

- Most values are within 60 to 85%, with a median around 75%.
- Outliers below 40% may suggest immunocompromised patients.
- The right-skewed distribution indicates higher neutrophil counts are common in suspected appendicitis cases.

5.2.12 Boxplot of RBC_Count (Red Blood Cell Count)

- RBC values are typically between 4.5 and 5.5, with a median of 5.
- A few low outliers suggest anemia, while extreme values beyond 14 indicate potential data issues.
- The distribution appears normal with minimal skewness.

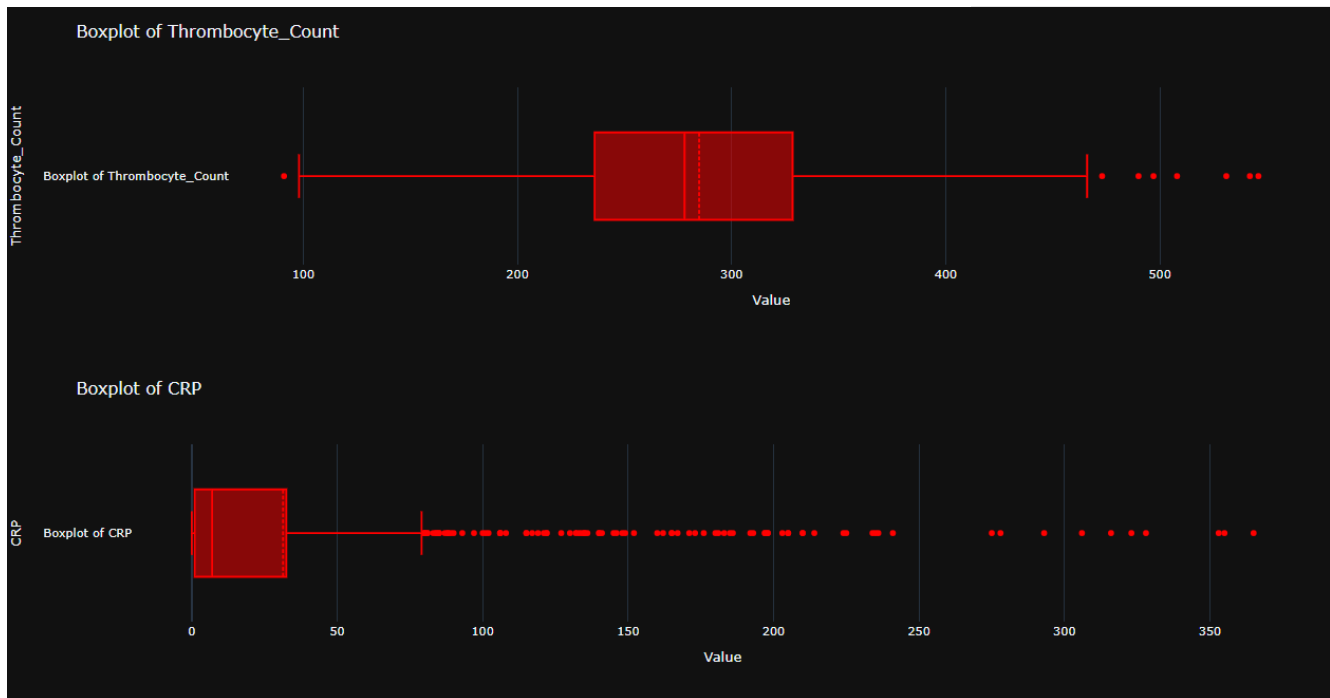


5.2.13 Boxplot of Hemoglobin

- Hemoglobin levels range from 12 to 15, with a median of around 14.
- Several outliers below 10 suggest anemia, while high values beyond 17 may indicate polycythemia.
- The distribution appears balanced but includes extreme values that may need attention.

5.2.14 Boxplot of RDW (Red Cell Distribution Width)

- Most RDW values lie between 12 and 15, with a median of 13.5.
- Extreme outliers beyond 20, and values reaching 90, indicate potential erroneous entries or severe medical conditions.
- The data shows a right skew with few extreme cases.



5.2.15 Boxplot of Thrombocyte_Count

- Thrombocyte counts are primarily within 200 to 350, with a median around 300.
- Several outliers above 450 may indicate thrombocytosis.
- The distribution is relatively symmetric with a few high values.

5.2.16 Boxplot of CRP (C-reactive Protein)

- CRP values show a heavily skewed distribution, with most values below 50, and a median around 20.
- Numerous extreme outliers beyond 200, indicating inflammatory conditions.
- The presence of multiple high values suggests a strong association with infection severity.

5.2.17 Overall Insights

5.2.17.1 **Outliers Detection:** Several variables, including **CRP**, **RDW**, **WBC_Count**, and **Length_of_Stay**, contain significant outliers, indicating possible extreme conditions or data anomalies.

5.2.17.2 **Data Distribution:** Most variables show a normal or right-skewed distribution, with a few variables such as CRP and RDW exhibiting strong skewness.

5.2.17.3 Potential Actions:

- Addressing outliers through capping or transformation.
- Investigating data entry errors for extreme values.
- Normalizing skewed distributions to enhance model performance.

5.3 Visualizing Numerical Data Using Histograms

The histograms of various numerical features in the dataset provide insights into their distribution, central tendencies, and potential outliers. Each variable is analyzed below in sequence:

```
# Assuming 'df' is your DataFrame
numerical_columns = df.select_dtypes(include=["number"]).columns

for col in numerical_columns:
    # Create the histogram
    fig = go.Figure()

    # Drop NaN values
    col_data = df[col].dropna()

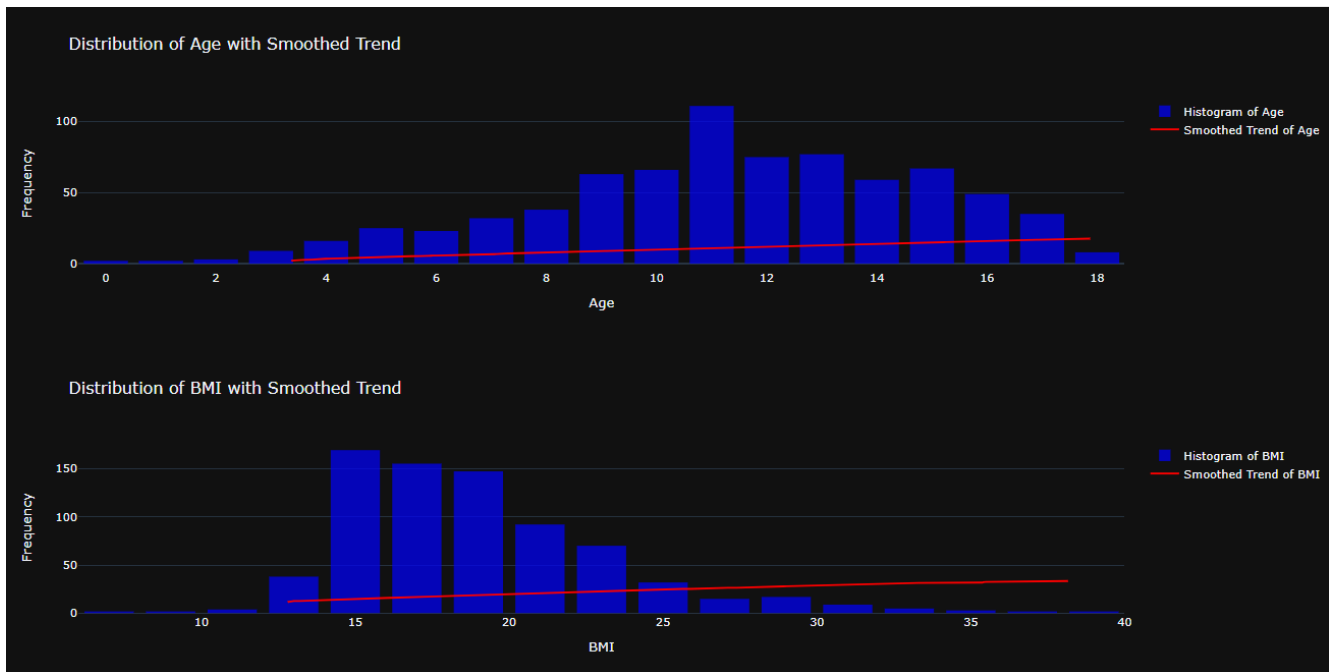
    # Add histogram trace
    fig.add_trace(go.Histogram(
        x=col_data,
        nbinsx=30,
        name=f"Histogram of {col}",
        opacity=0.7,
        marker=dict(color='blue')
    ))

    # Compute a smoothed trend using a rolling average
    sorted_col_data = np.sort(col_data) # Sort data for a smooth curve
    rolling_window = max(len(sorted_col_data) // 50, 5) # Rolling window size (5 or 2% of data)
    rolling_avg = pd.Series(sorted_col_data).rolling(rolling_window).mean()

    # Add smoothed trend trace
    fig.add_trace(go.Scatter(
        x=sorted_col_data[len(sorted_col_data) - len(rolling_avg.dropna()):], # Adjust x-axis to match rolling average
        y=rolling_avg.dropna(),
        mode='lines',
        name=f"Smoothed Trend of {col}",
        line=dict(color='red', width=2)
    ))

    # Update Layout
    fig.update_layout(
        title=f"Distribution of {col} with Smoothed Trend",
        xaxis_title=col,
        yaxis_title="Frequency",
        template="plotly_dark", # Optional: for a dark theme
        bargap=0.2, # Space between bars
    )

    # Show the plot
    fig.show()
```

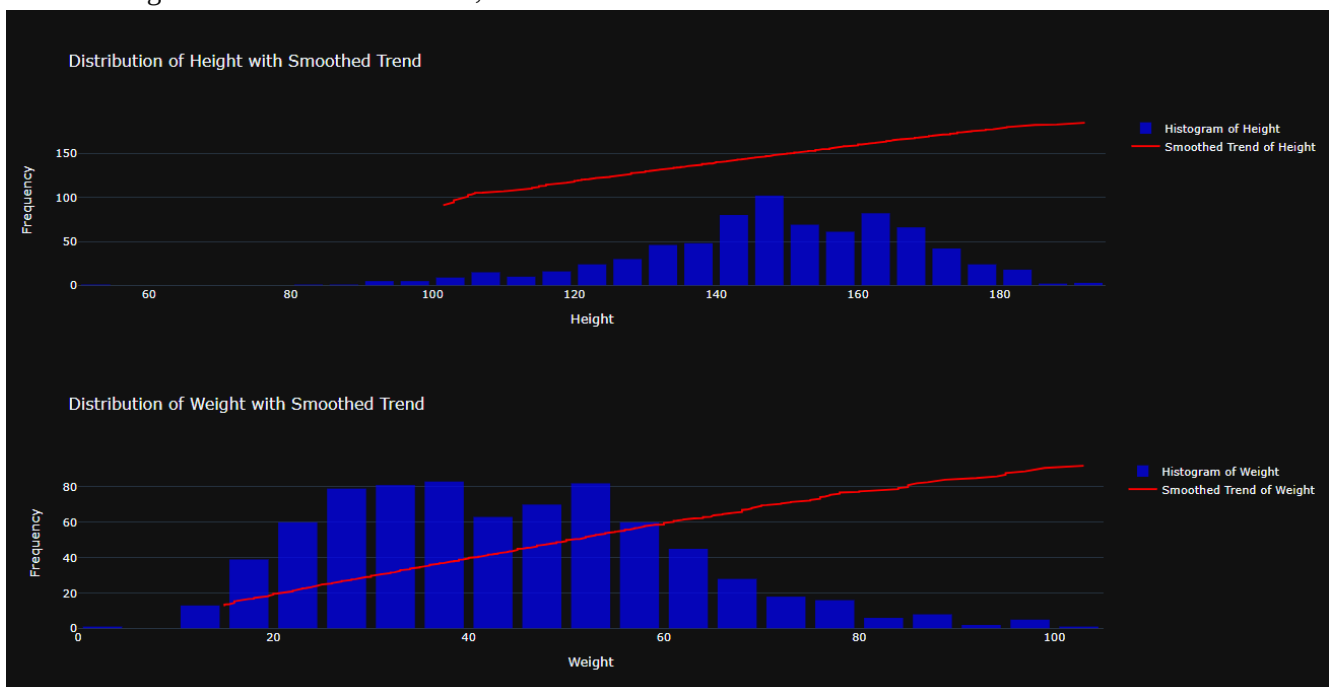


5.3.1 Age

- Distribution: Right-skewed, with most patients between 8 and 15 years and a peak around 11-12 years.
- Outliers: Younger patients (<6 years) and older adolescents (>16 years) are rare.
- Insight: Most patients fall in the pre-adolescent age group, aligning with the expected population for appendicitis.

5.3.2 BMI (Body Mass Index)

- Distribution: Normal-like, with the majority of values between 15 and 25, peaking around 20.
- Outliers: Higher BMI values (>30) indicate overweight or obese cases.
- Insight: BMI is well-distributed, with few extreme cases.

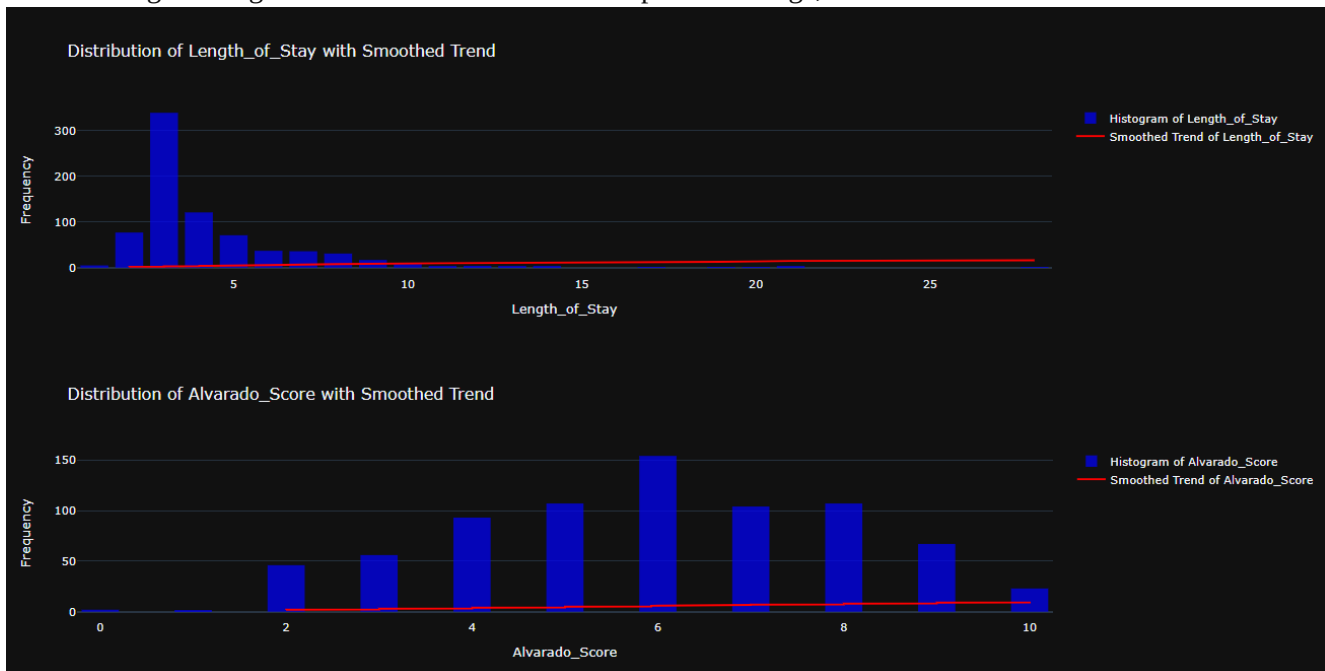


5.3.3 Height

- Distribution: Normal, centered between 140 and 170 cm, peaking around 160 cm.
- Outliers: Very short patients (<100 cm) are rare.
- Insight: The height distribution aligns with average pediatric measurements.

5.3.4 Weight

- Distribution: Slightly right-skewed, with most values between 30 and 60 kg and a peak around 40-50 kg.
- Outliers: Heavier patients (>80 kg) are infrequent.
- Insight: Weight is concentrated in the normal pediatric range, with few extreme values.

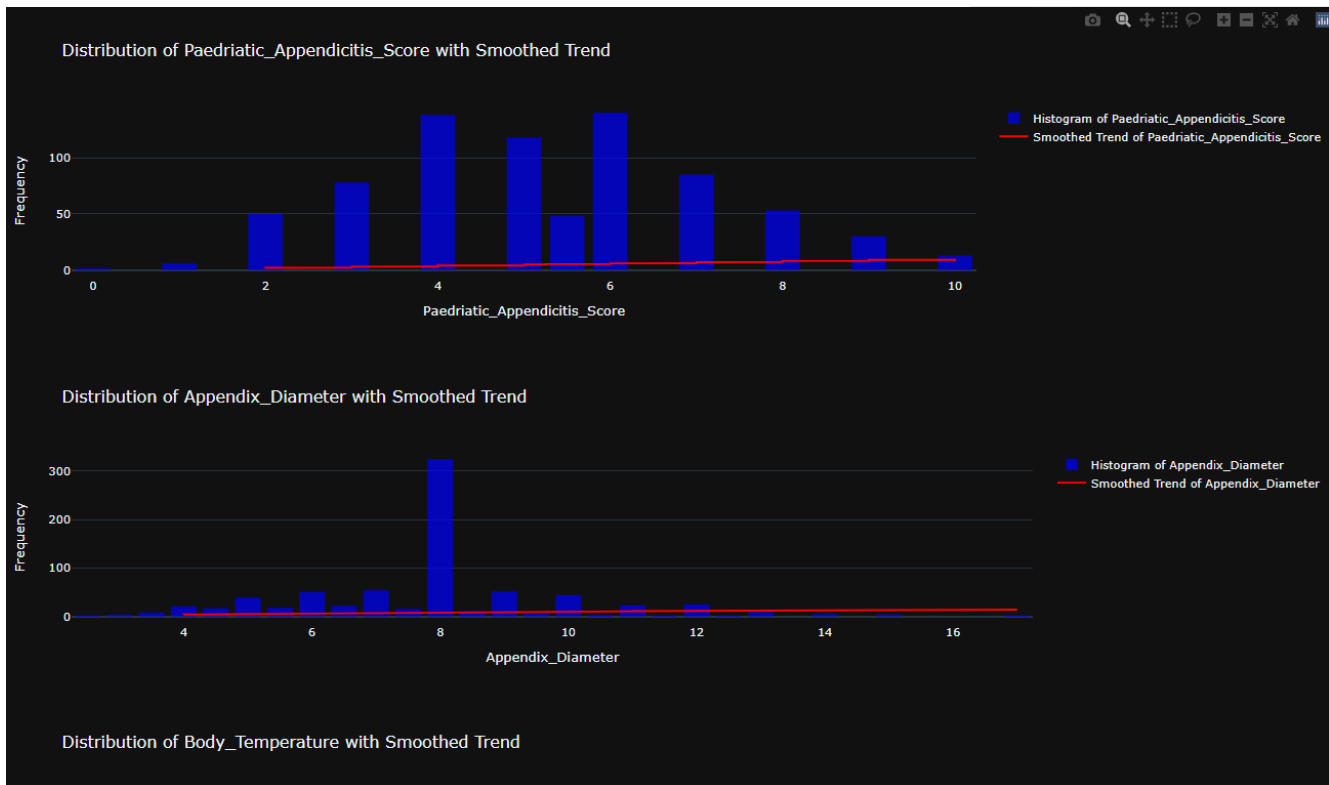


5.3.5 Length_of_Stay

- Distribution: Right-skewed, with most stays within 1 to 5 days, peaking around 3 days.
- Outliers: Stays longer than 10 days are rare.
- Insight: Most patients are discharged within a week, with extended stays being exceptional.

5.3.6 Alvarado_Score

- Distribution: Normal-like, with scores concentrated between 4 and 8, peaking at 6.
- Outliers: Very low (<2) and maximum (10) scores are uncommon.
- Insight: The distribution aligns with moderate-to-high appendicitis risk.

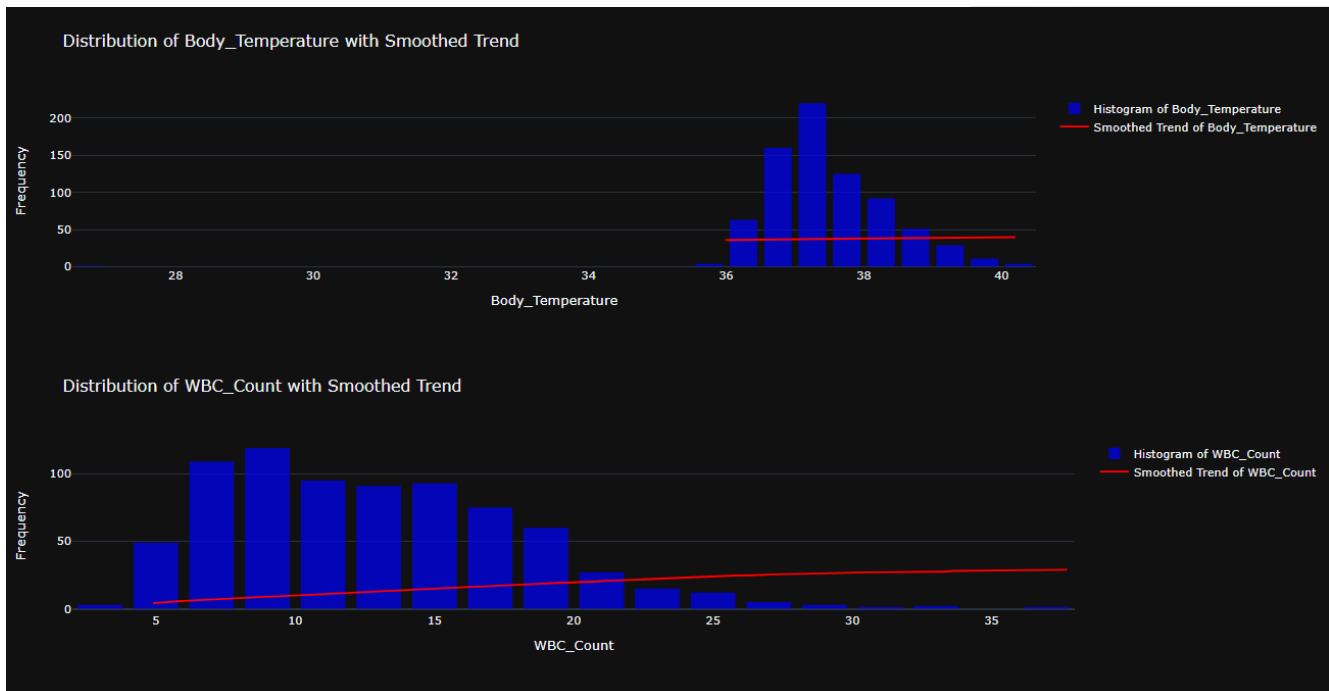


5.3.7 Paediatric_Appendicitis_Score

- Distribution: Normal-like, centered between 4 and 6, peaking around 5.
- Outliers: Scores at the extremes (0 and 10) are rare.
- Insight: Most patients fall in the moderate-risk category for appendicitis.

5.3.8 Appendix_Diameter

- Distribution: Right-skewed, with most values around 7-8 mm.
- Outliers: Diameters above 10 mm suggest severe inflammation.
- Insight: Typical appendix size aligns with diagnostic thresholds for appendicitis.

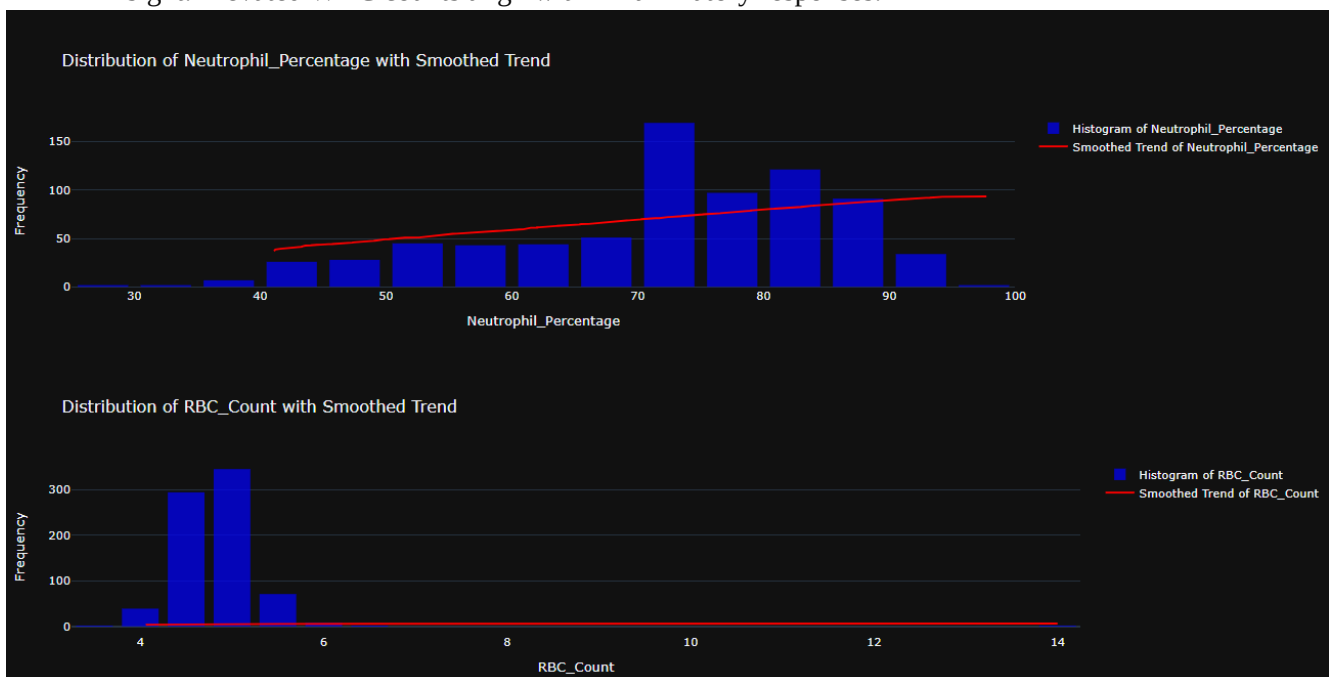


5.3.9 Body_Temperature

- Distribution: Normal, centered between 36°C and 38°C, peaking around 37°C.
- Outliers: Temperatures above 38°C indicate fever, while values below 30°C may be errors.
- Insight: Elevated temperatures correlate with infection or inflammation.

5.3.10 WBC_Count (White Blood Cell Count)

- Distribution: Right-skewed, with most values between 7 and 15, peaking around 10.
- Outliers: Extreme values (>30) suggest severe infections.
- Insight: Elevated WBC counts align with inflammatory responses.

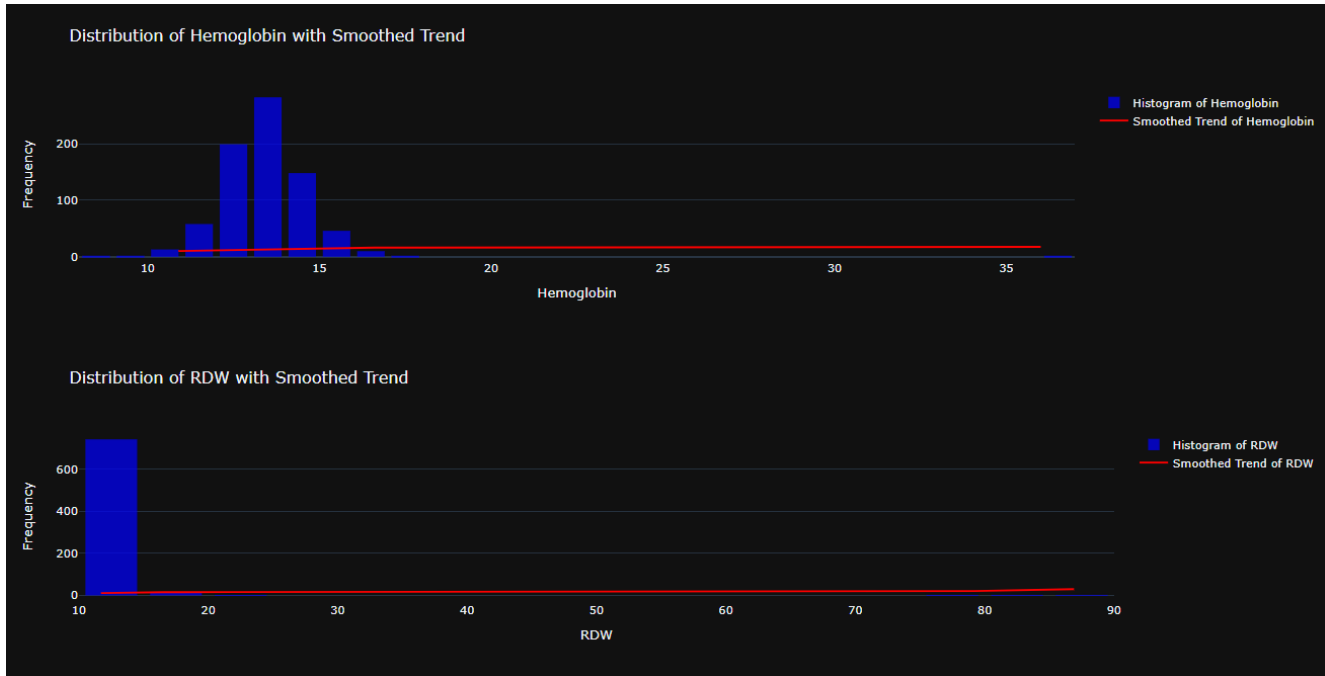


5.3.11 Neutrophil_Percentage

- Distribution: Normal-like, concentrated between 50% and 90%, peaking around 70%.
- Outliers: Low values (<40%) are rare.
- Insight: Elevated neutrophil percentages are consistent with appendicitis-associated inflammation.

5.3.12 RBC_Count (Red Blood Cell Count)

- Distribution: Normal, centered around 5, with most values between 4 and 5.5.
- Outliers: Extremely high (>6) or low (<4) values are rare.
- Insight: RBC counts show minimal variability, with rare deviations indicating anemia or polycythemia.

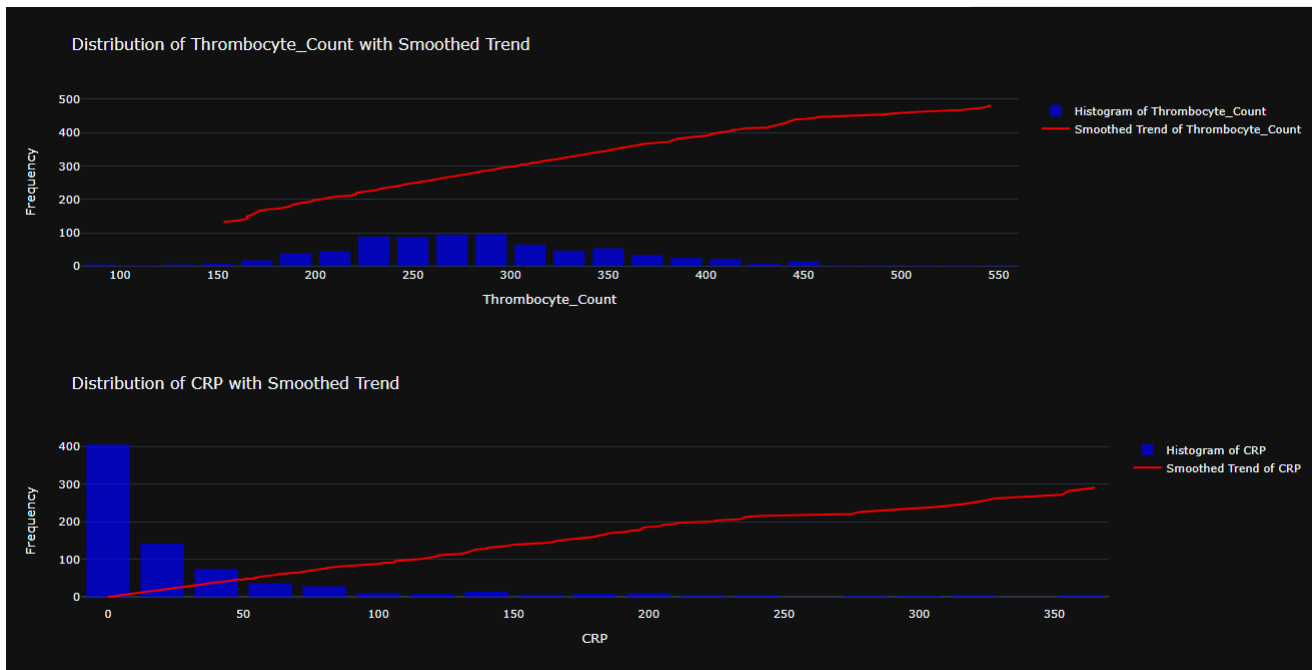


5.3.13 Hemoglobin

- Distribution: Normal, centered between 12 and 16 g/dL, peaking at 14 g/dL.
- Outliers: Low (<10 g/dL) and very high (>20 g/dL) values are uncommon.
- Insight: The majority exhibit normal hemoglobin levels, with rare deviations suggesting anemia.

5.3.14 RDW (Red Cell Distribution Width)

- Distribution: Right-skewed, concentrated between 12% and 15%, with a sharp peak.
- Outliers: Values above 20% suggest hematological abnormalities.
- Insight: RDW is concentrated in the normal range, with outliers indicating potential conditions like anemia.



5.3.15 Thrombocyte_Count

- Distribution: Normal-like, with most values between 200 and 350, peaking at 300.
- Outliers: Elevated counts (>450) suggest thrombocytosis.
- Insight: Platelet counts are well-distributed, with few deviations.

5.3.16 CRP (C-reactive Protein)

- Distribution: Heavily right-skewed, with most values below 50 mg/L, peaking at 0-10 mg/L.
- Outliers: Extreme values (>300 mg/L) indicate severe inflammation.
- Insight: CRP is a strong marker for infection or inflammation, with a wide variability.

5.3.17 Overall Insights

- Most features exhibit normal or slightly skewed distributions, with some outliers indicating severe medical conditions or data anomalies.
 - Features like **CRP**, **WBC_Count**, **Neutrophil_Percentage**, and **Appendix_Diameter** show significant variability and are key markers of inflammation or infection, making them highly relevant for appendicitis prediction.
 - Handling outliers and ensuring data quality are essential steps for robust modeling and analysis.
- These histograms collectively validate the dataset's quality and clinical relevance, providing a solid foundation for predictive modeling and feature engineering.

5.4 Visualizing Categorical Data Using Bar Plots

A **bar plot** (or **bar chart**) is a graphical representation used to display and compare categorical data. It consists of rectangular bars, where the length or height of each bar represents the frequency or proportion of a category.

```
# Assuming 'df' is your DataFrame
numerical_columns = df.select_dtypes(include=["number"]).columns

for col in numerical_columns:
    # Create the histogram
    fig = go.Figure()

    # Drop NaN values
    col_data = df[col].dropna()

    # Add histogram trace
    fig.add_trace(go.Histogram(
        x=col_data,
        nbinsx=30,
        name=f"Histogram of {col}",
        opacity=0.7,
        marker=dict(color='blue')
    ))

    # Compute a smoothed trend using a rolling average
    sorted_col_data = np.sort(col_data) # Sort data for a smooth curve
    rolling_window = max(len(sorted_col_data) // 50, 5) # Rolling window size (5 or 2% of data)
    rolling_avg = pd.Series(sorted_col_data).rolling(rolling_window).mean()

    # Add smoothed trend trace
    fig.add_trace(go.Scatter(
        x=sorted_col_data[len(sorted_col_data) - len(rolling_avg.dropna()):], # Adjust x-axis to match rolling average
        y=rolling_avg.dropna(),
        mode='lines',
        name=f"Smoothed Trend of {col}",
        line=dict(color='red', width=2)
    ))

    # Update layout
    fig.update_layout(
        title=f"Distribution of {col} with Smoothed Trend",
        xaxis_title=col,
        yaxis_title="Frequency",
        template="plotly_dark", # Optional: for a dark theme
        bargap=0.2, # Space between bars
    )

    # Show the plot
    fig.show()
```




5.4.1 Distribution of Sex

- Male: 395 cases.
- Female: 364 cases.

Insight:

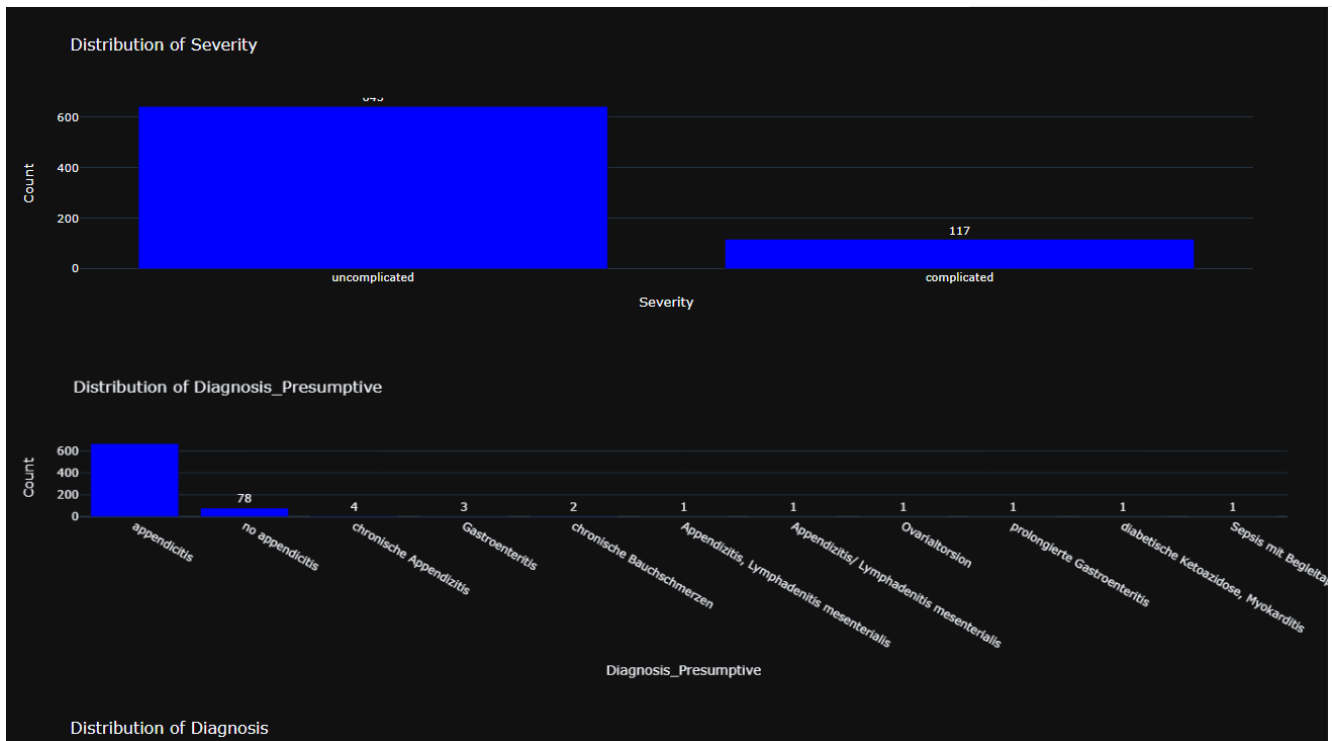
- There is a nearly equal distribution of males and females, with males being slightly more prevalent in this dataset.

5.4.2 Distribution of Management

- Conservative: 476 cases.
- Primary Surgical: 265 cases.
- Secondary Surgical: 18 cases.

Insight:

- Most cases are managed conservatively, with surgical management (primary and secondary) being less frequent, indicating that conservative treatments are more common in this context.



5.4.3 Distribution of Severity

- Uncomplicated: 645 cases.
- Complicated: 117 cases.

Insight:

- The majority of cases are uncomplicated, suggesting that severe complications are relatively rare in this dataset.

5.4.4 Distribution of Diagnosis_Presumptive

- Appendicitis: 589 cases.
- No Appendicitis: 78 cases.
- Other categories (e.g., Gastroenteritis, Chronische Bauchschmerzen) have very few cases.

Insight:

- Most cases are presumed to be appendicitis, making it the dominant diagnosis, with a few cases of other conditions.



5.4.5 Distribution of Diagnosis

- Appendicitis: 475 cases.
- No Appendicitis: 314 cases.

Insight:

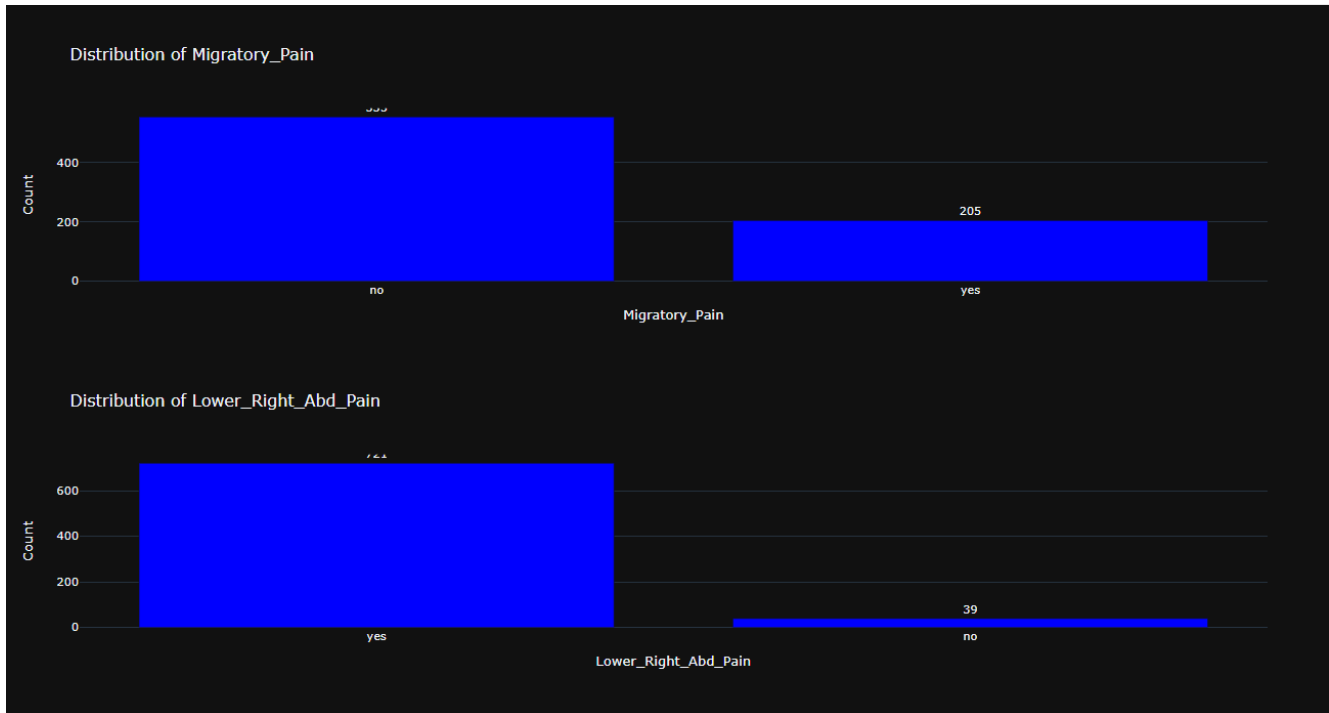
- Appendicitis is more frequently confirmed, but a significant number of cases are diagnosed as something else.

5.4.6 Distribution of Appendix_on_US

- Yes: 475 cases.
- No: 263 cases.

Insight:

- A majority of cases had an appendix visible on ultrasound (US), which is likely a key diagnostic tool.



5.4.7 Distribution of Migratory_Pain

- No: 554 cases.
- Yes: 205 cases.

Insight:

- Migratory pain is absent in most cases, suggesting it is not a universal symptom.

5.4.8 Distribution of Lower_Right_Abd_Pain

- Yes: 641 cases.
- No: 39 cases.

Insight:

- Lower-right abdominal pain is highly prevalent and likely a key diagnostic feature.



5.4.9 Distribution of Contralateral_Rebound_Tenderness

- No: 466 cases.
- Yes: 292 cases.

Insight:

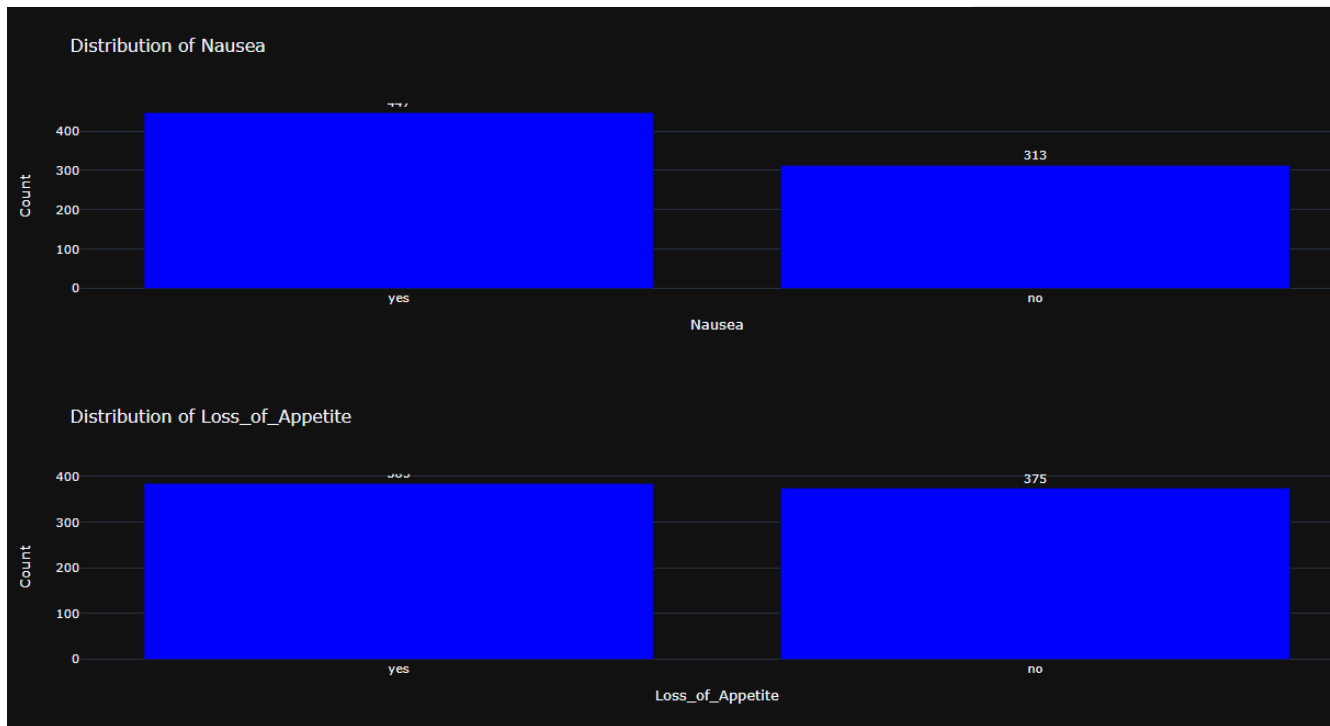
- Rebound tenderness is present in a significant portion of cases, making it a notable symptom.

5.4.10 Distribution of Coughing_Pain

- No: 547 cases.
- Yes: 211 cases.

Insight:

- Coughing pain is not present in the majority but is still a relevant symptom in many cases.



5.4.11 Distribution of Nausea

- Yes: 477 cases.
- No: 313 cases.

Insight:

- Nausea is a common symptom, observed in more than half the cases.

5.4.12 Distribution of Loss_of_Appetite

- Yes: 508 cases.
- No: 375 cases.

Insight:

- Loss of appetite is prevalent in the dataset, seen in the majority of cases.



5.4.13 Distribution of Neutrophilia

- No: 405 cases.
- Yes: 354 cases.

Insight:

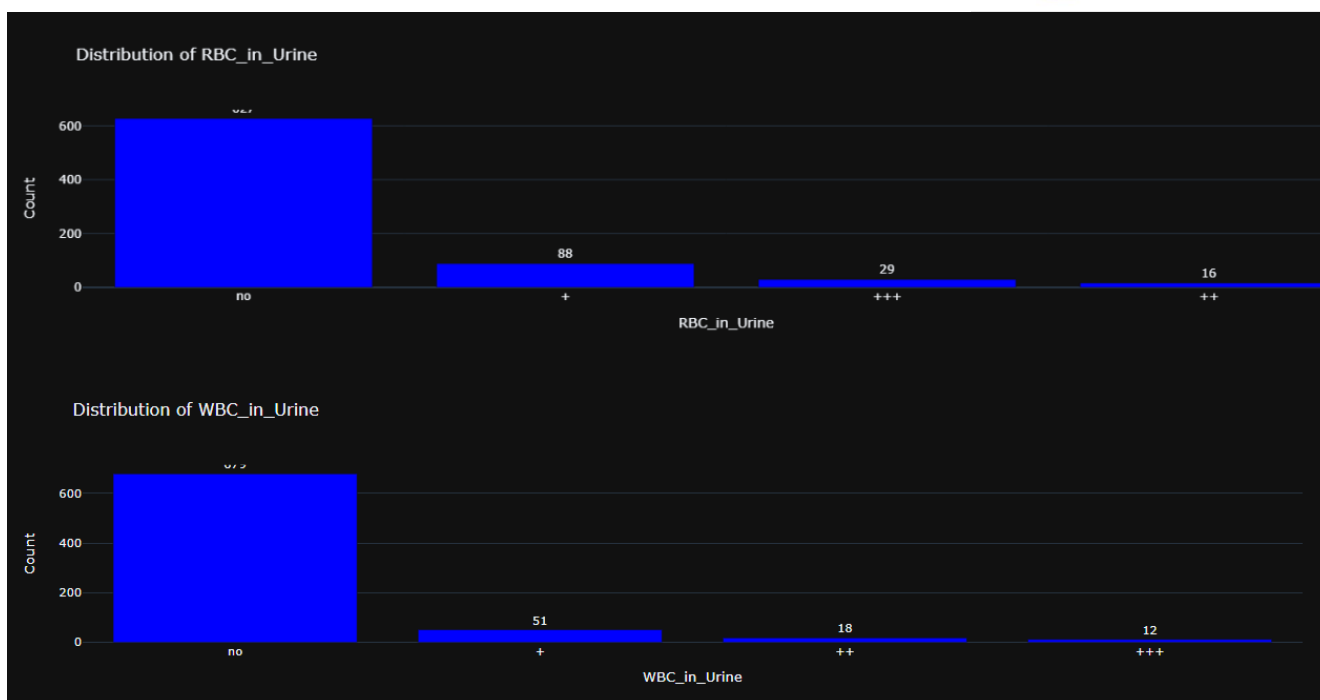
- Neutrophilia is relatively balanced between presence and absence, indicating its variable diagnostic value.

5.4.14 Distribution of Ketones_in_Urine

- No: 510 cases.
- Yes (+++, ++, +): 244 cases (combined).

Insight:

- The absence of ketones is more common, but a significant portion of cases show varying levels of ketones.



5.4.15 Distribution of RBC_in_Urine

- No: 606 cases.
- Yes (+, ++, +++): 133 cases (combined).

Insight:

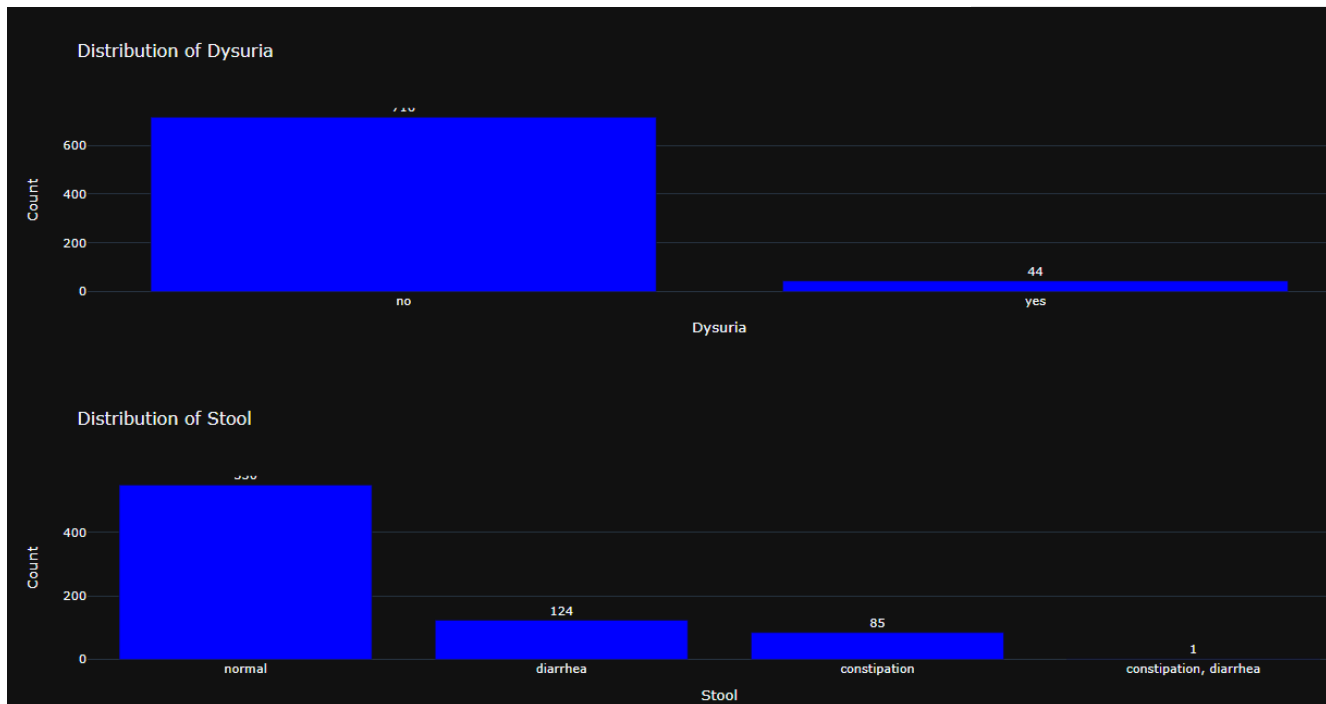
- Most cases do not show RBC in urine, making it less frequent as a diagnostic marker.

5.4.16 Distribution of WBC_in_Urine

- No: 589 cases.
- Yes (+, ++, +++): 81 cases (combined).

Insight:

- WBC in urine is even less common than RBC, suggesting it is an infrequent diagnostic finding.



5.4.17 Distribution of Dysuria

- No: 710 cases.
- Yes: 44 cases.

Insight:

- **Dysuria is rare in this dataset.**

5.4.18 Distribution of Stool

- Normal: 556 cases.
- Diarrhea: 124 cases.
- Constipation: 85 cases.
- Constipation + Diarrhea: 1 case.

Insight:

- Normal stool is observed in the majority of cases, with diarrhea and constipation being less common.



5.4.19 Distribution of Peritonitis

- No: 390 cases.
- Local: 186 cases.
- Generalized: 40 cases.

Insight:

- Peritonitis is not observed in most cases, but local peritonitis is more common than generalized.

5.4.20 Distribution of Psoas_Sign

- No: 550 cases.
- Yes: 230 cases.

Insight:

- Psoas sign is present in a significant minority, indicating its role as a diagnostic sign.



5.4.21 Distribution of Ipsilateral_Rebound_Tenderness

- No: 456 cases.
- Yes: 38 cases.

Insight:

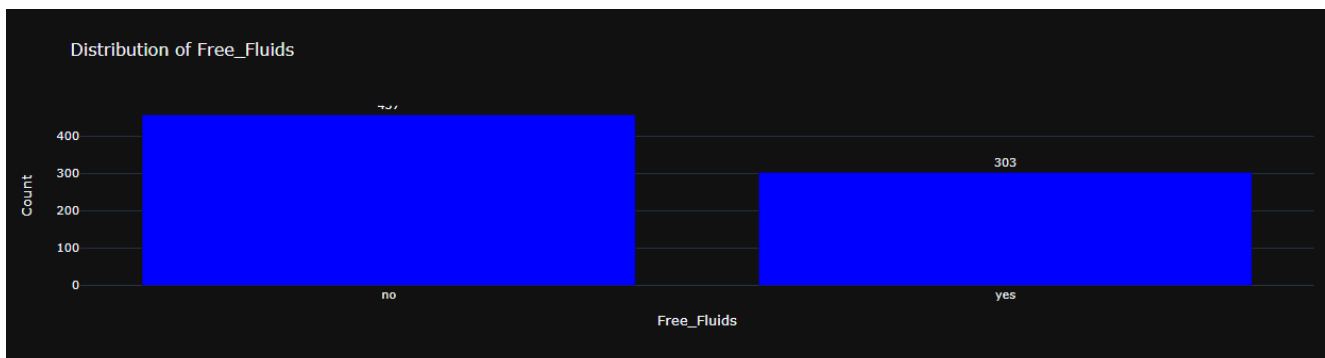
- Ipsilateral rebound tenderness is relatively rare.

5.4.22 Distribution of US_Performed

- Yes: 745 cases.
- No: 12 cases.

Insight:

- Ultrasound is performed in almost all cases, indicating its importance in diagnosis.



5.4.23 Distribution of Free_Fluids

- No: 456 cases.
- Yes: 303 cases.

Insight:

- The presence of free fluids is observed in a significant number of cases, though it is slightly less common than its absence.

General Observations:

- Some categorical features like Lower_Right_Abd_Pain, Nausea, and Loss_of_Appetite are highly prevalent.
- Diagnostic features like Appendix_on_US, Peritonitis (Local), and Free_Fluids play an essential role in diagnosis.
- Symptoms like Ipsilateral_Rebound_Tenderness and Dysuria are less frequent, suggesting limited diagnostic use in this dataset.

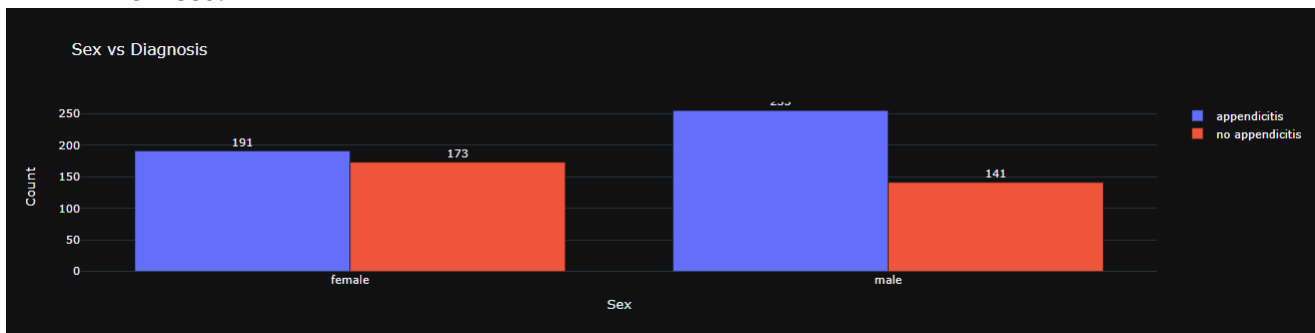
5.5 Analysis of Factors Associated with Appendicitis Diagnosis

5.5.1 Sex vs. Diagnosis

- Observation: Males have a slightly higher count of appendicitis cases (229) compared to females (191). However, in the "no appendicitis" category, females (173) are higher than males (141).

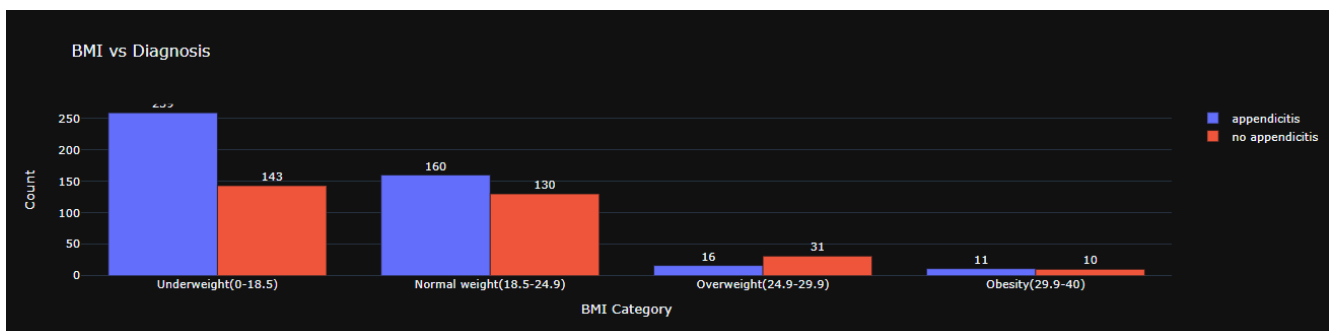
Insight:

- There might be a slight gender-based variation in appendicitis diagnosis, with males having a higher likelihood.



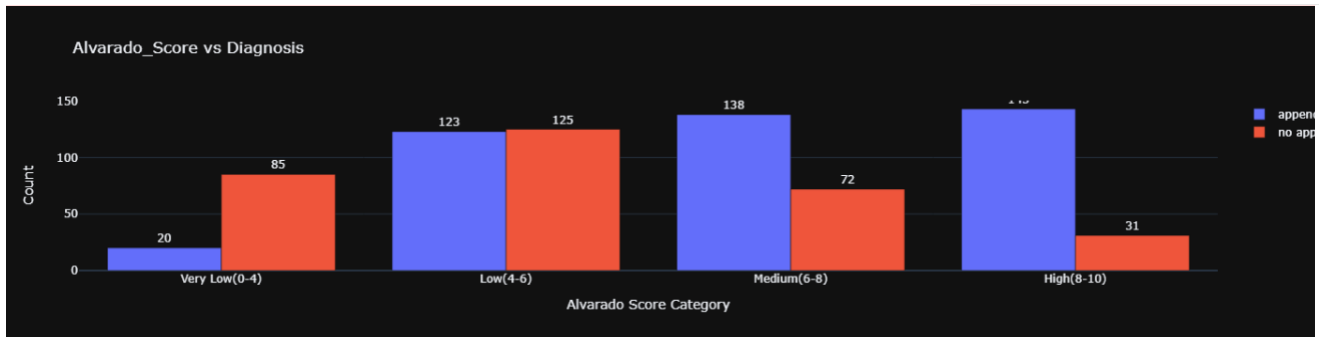
5.5.2 BMI vs. Diagnosis

- The majority of appendicitis cases occur in the underweight BMI category (243). Normal weight individuals follow with 160 cases. Overweight and obesity categories have significantly fewer cases.
- **Insight:** Underweight individuals might be more prone to appendicitis, or there could be an underlying data bias.



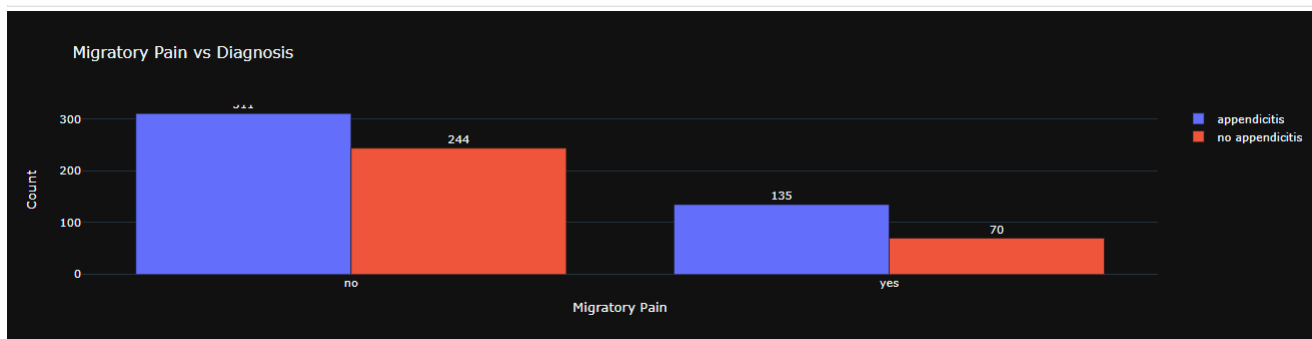
5.5.3 Alvarado Score vs. Diagnosis

- A significant number of appendicitis cases are in the medium (138) and high (143) Alvarado score categories. Cases in the "very low" category are very few for appendicitis.
- **Insight:** The Alvarado score aligns well with appendicitis diagnosis, supporting its diagnostic utility.



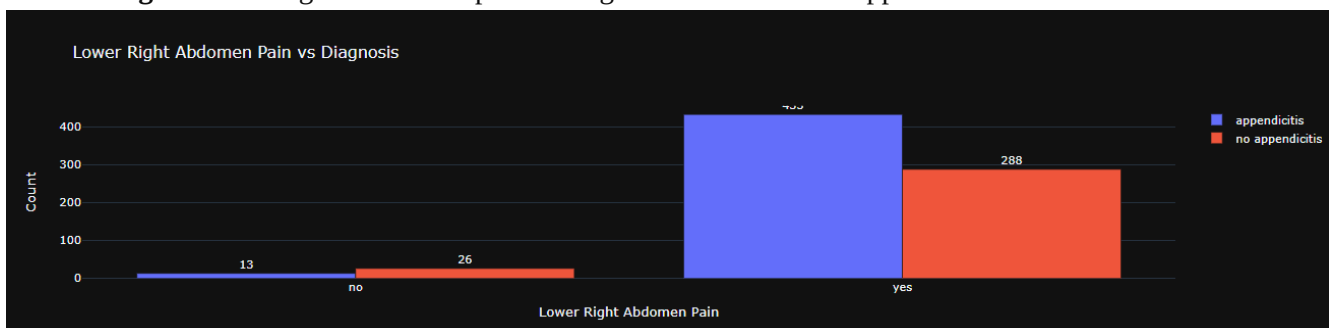
5.5.4 Migratory Pain vs. Diagnosis

- Most appendicitis cases (311) reported no migratory pain, while a smaller proportion (135) had migratory pain. The trend is similar for "no appendicitis" cases but with reduced counts.
- Insight:** Migratory pain might not be a predominant symptom for appendicitis.



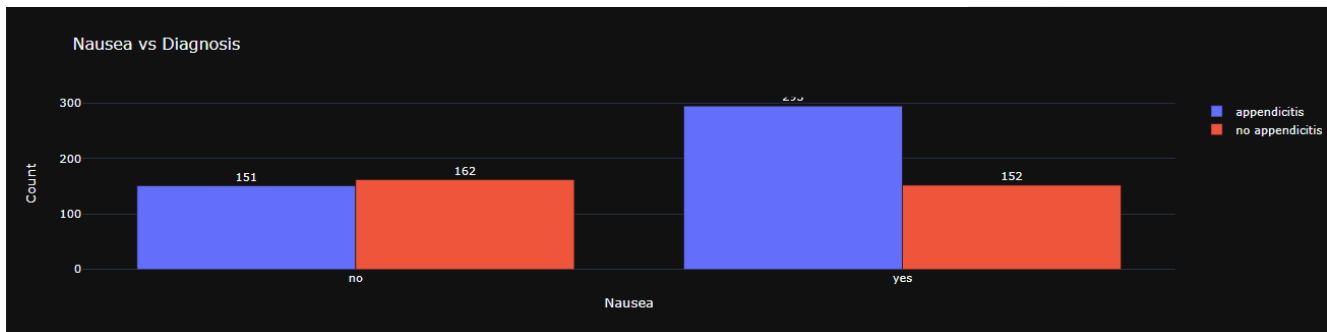
5.5.5 Lower Right Abdomen Pain vs. Diagnosis

- A majority of appendicitis cases (393) report lower-right abdominal pain, compared to only 13 cases without the pain. The trend is similar but reversed for the "no appendicitis" group.
- Insight:** Lower-right abdominal pain is a significant indicator of appendicitis.



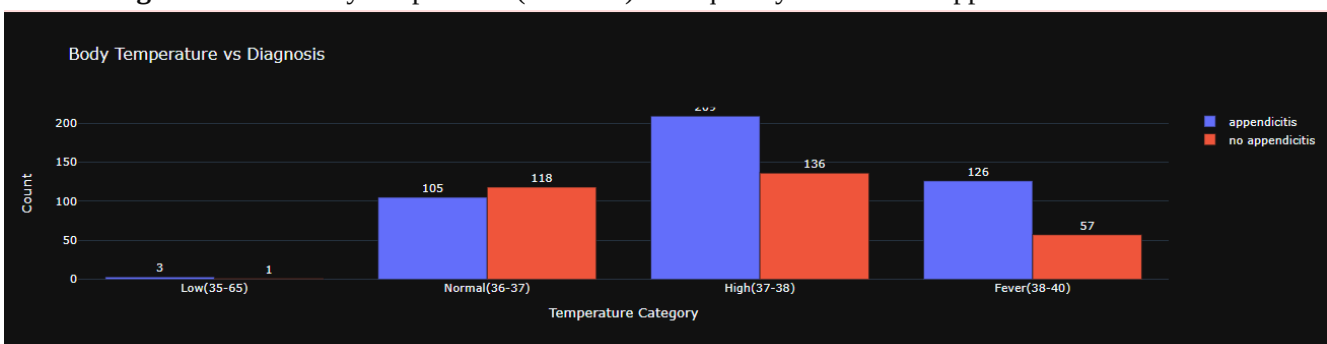
5.5.6 Nausea vs. Diagnosis

- Appendicitis cases are higher in individuals reporting nausea (259) than those without (151). A similar trend exists for "no appendicitis" cases but with a lower overall count.
- Insight:** Nausea appears to be a common symptom associated with appendicitis.



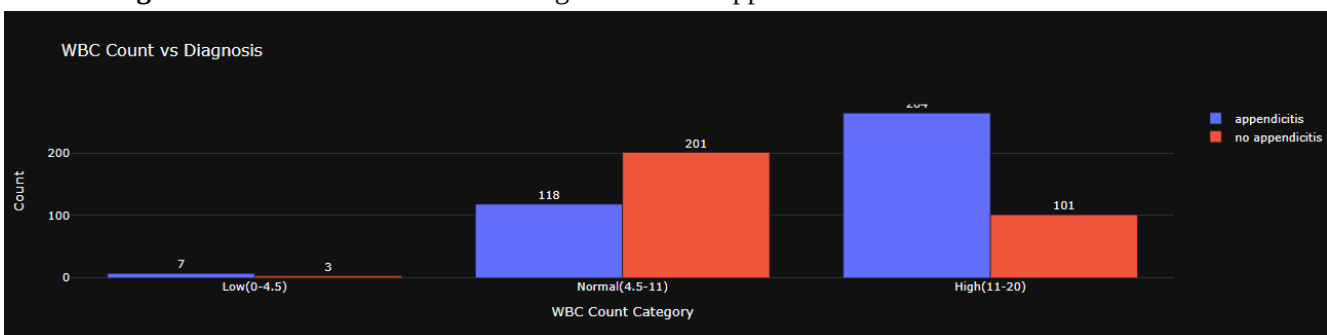
5.5.7 Body Temperature vs. Diagnosis

- Most appendicitis cases are in the "high temperature" (37-38°C) category, followed by "normal temperature." Few cases are observed in "low" and "fever" ranges.
- Insight:** Elevated body temperature (37-38°C) is frequently observed in appendicitis cases.



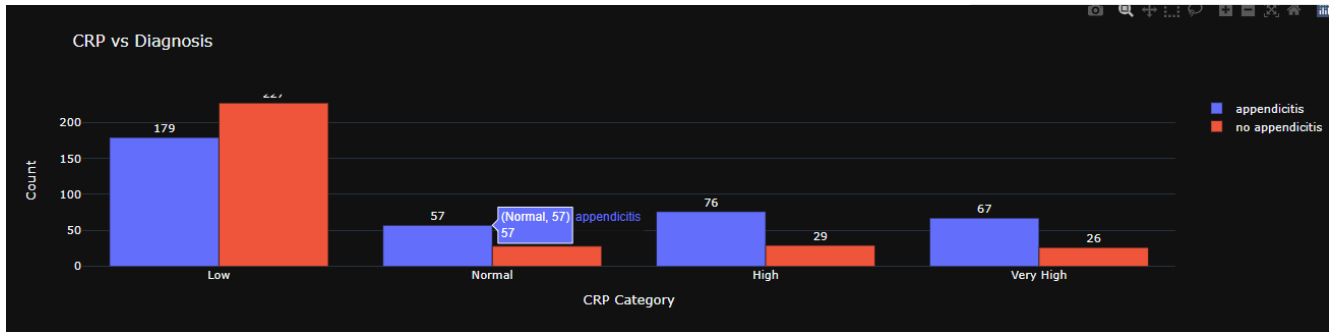
5.5.8 WBC Count vs. Diagnosis

- Appendicitis cases are more frequent in the "high WBC count" (11-20) category, followed by "normal WBC count." Very few cases occur in the "low WBC count" range.
- Insight:** Elevated WBC count is a strong indicator of appendicitis.



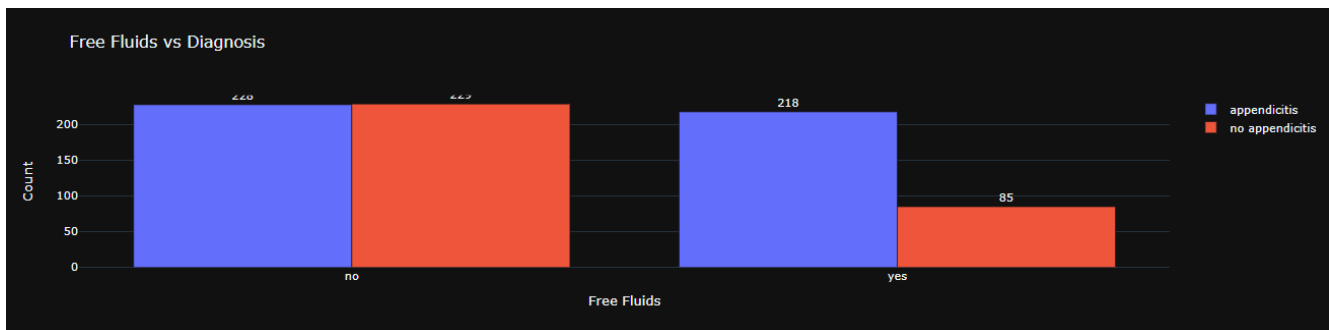
5.5.9 CRP vs. Diagnosis

- The majority of appendicitis cases are in the "low CRP" category (179), but significant counts are observed in "high" (76) and "very high" (67) CRP categories.
- Insight:** Elevated CRP levels might support the diagnosis of appendicitis in certain cases.



5.5.10 Free Fluids vs. Diagnosis

- Appendicitis cases are equally distributed across "no free fluids" (228) and "free fluids present" (218). The "no appendicitis" group has fewer cases in the "free fluids" category.
- Insight:** Free fluids might have a moderate association with appendicitis diagnosis but are not definitive.



6 Feature Extraction Using Pre-trained ResNet-50

This code modifies a **pre-trained ResNet-50 model** to use it as a **feature extractor** instead of a classification model.

6.1 Load a Pre-trained ResNet-50 Model

- models.resnet50(pretrained=True):** Loads a ResNet-50 model with weights pre-trained on the **ImageNet dataset** (a large image dataset with 1,000 classes).
- Pre-training allows the model to extract **generic features** from images that can be transferred to other tasks (transfer learning).

6.2 Set the Model to Evaluation Mode

- Purpose:** Prepares the model for **inference** by:
- Disabling **dropout layers** (used only during training).
- Freezing the **batch normalization statistics** to ensure stable behavior.
- This mode is crucial for consistent and efficient feature extraction.

6.3 Remove the Final Fully Connected Layer

- **Why Remove the Final Layer?**

- The final layer of ResNet-50 is a **fully connected (FC) layer** with 1,000 output neurons designed for ImageNet classification.
- Since we want **feature embeddings** (not class probabilities), we remove this classification layer.

➤ **How It Works:**

- `model.children()`: Gets a list of all the layers in ResNet-50.
- `[:-1]`: Excludes the last layer (the FC classification layer).
- `torch.nn.Sequential(...)`: Reconstructs the model without the FC layer.

6.4 Transformation Pipeline:

This pipeline preprocesses an image to ensure compatibility with a **pre-trained**

ResNet-50 model:

```
transform = transforms.Compose([
    transforms.ToPILImage(),
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
```

1. **`transforms.ToPILImage()`** → Converts the image to a **PIL format**.
2. **`transforms.Resize((224, 224))`** → Resizes the image to **224 × 224 pixels**, the input size required by ResNet-50.
3. **`transforms.ToTensor()`** → Converts the image to a **PyTorch tensor**, scaling pixel values to the range [0, 1].
4. **`transforms.Normalize(mean, std)`** → Normalizes the pixel values using the **mean and standard deviation** of the ImageNet dataset.

Purpose:

- Prepares the image to match the **expected input format** for ResNet-50.
- Ensures uniformity in size, format, and pixel intensity for optimal performance during inference or training.

6.5 Feature Extraction

This function **extracts deep features** from an image using a **pre-trained ResNet-50 model**.

```
# Function to extract image features using ResNet
def extract_features(image_path):
    # Read the image
    image = cv2.imread(image_path)
    if image is None:
        print(f"Warning: Unable to read the image {image_path}")
        return None
    # Convert the image from BGR to RGB
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    # Apply the transformation
    image = transform(image).unsqueeze(0) # Add batch dimension
    if image.dim() != 4: # Check if the image has the expected dimensions (batch, channels, height, width)
        print('test')
        print(f"Warning: Unexpected dimensions for image {image_path}")
        return None

    # Extract features
    with torch.no_grad():
        features = model(image)

    features = features.squeeze().numpy()
    #print(features)
    return features
```

6.5.1 Read the Image:

- `cv2.imread(image_path)`: Loads the image using OpenCV.
- If the image cannot be read, it prints a warning and returns `None`.

6.5.2 Convert BGR to RGB:

- OpenCV loads images in **BGR format**, but PyTorch expects **RGB**.
- `cv2.cvtColor(image, cv2.COLOR_BGR2RGB)` corrects this.

6.5.3 Apply Preprocessing (Transformations):

- Uses the **previously defined transform pipeline**.
- `.unsqueeze(0)`: Adds a **batch dimension** since ResNet expects batch input.
- **Checks for dimension mismatches** and returns `None` if incorrect.

6.5.4 Extract Features using ResNet-50:

- **Disables gradient computation** (`torch.no_grad()`) for efficiency.
- Feeds the image into the **ResNet-50 model** (without the last classification layer).
- Extracts **feature embeddings** (high-level numerical representation of the image).
- Converts the extracted features into a **NumPy array** for further processing.

Purpose:

- Converts an image into a **2048-dimensional feature vector** using ResNet.
- These features can be used for tasks like **classification, clustering, or similarity analysis**.

7 Preprocessing:

7.1 Parsing Us_Number

This function extracts a numeric identifier (US_Number) from the beginning of a file name.

```
# Function to parse the US_Number from the image file name
def parse_us_number(filename):
    match = re.match(r'^(\d+)\.', filename)
    if match:
        return int(match.group(1))
    return None
```

7.1.1 Uses Regular Expressions (re.match)

- `re.match(r'^(\d+)\.', filename):`
 - Looks for **digits (d+)** at the **start (^)** of the filename.
 - The digits must be **followed by a dot (.)** to ensure correct extraction.
 - Example matches:
 - "123.image.jpg" → ☒ Extracts 123
 - "456_data.png" → ☐ Doesn't match (no . after number)

7.1.2 Extracts and Converts the Number

- If a match is found: `int(match.group(1))` returns the extracted **numeric value**.
- If no match is found: Returns `None`.

Purpose:

- Extracts **unique numeric identifiers** from image filenames.
- Useful for **sorting, indexing, or linking** images to database records.
- Ensures that only correctly formatted filenames are processed.

7.2 Processing and Storing Image Features

This script processes images, extracts their deep learning features using a ResNet model, and organizes them into a structured DataFrame.

```
us_number_to_features = {}

# Process all images and extract features
for image_file in os.listdir(image_dir):
    image_path = os.path.join(image_dir, image_file)
    us_number = parse_us_number(image_file)

    if us_number is not None:
        features = extract_features(image_path)
        if features is not None:
            if us_number not in us_number_to_features:
                us_number_to_features[us_number] = []
            us_number_to_features[us_number].append(features)

# Create a DataFrame to hold all features with dynamic column naming
all_features_df = pd.DataFrame()

for us_number, features_list in us_number_to_features.items():
    # Create a dictionary for storing feature columns for this US_Number
    feature_columns = {}
    for i, features in enumerate(features_list):
        # Add each feature vector to separate columns
        for j, feature_value in enumerate(features):
            column_name = f'Image_{i+1}_Feature_{j+1}'
            feature_columns[column_name] = feature_value
    # Convert to a DataFrame and add US_Number
    temp_df = pd.DataFrame([feature_columns])
    temp_df['US_Number'] = us_number
    # Append to the main DataFrame
    all_features_df = pd.concat([all_features_df, temp_df], ignore_index=True)
```

7.2.1 Initialize Storage:

us_number_to_features: A dictionary to map each US_Number to its list of extracted image feature vectors.

7.2.2 Process Images:

Loop through all images in image_dir.

Parse the US_Number from the filename using parse_us_number.

Extract features for each image using extract_features.

Group and store feature vectors for each US_Number.

7.2.3 Create a DataFrame:

For each US_Number and its associated feature vectors:

Dynamically create columns named as Image_X_Feature_Y for individual feature values.

Store these features in a temporary DataFrame, including the US_Number.

Append all temporary DataFrames into a single all_features_df.

7.2.4 Result:

all_features_df: A structured DataFrame containing:

US_Number as the identifier.

Dynamically named columns for all extracted features (e.g., Image_1_Feature_1, Image_1_Feature_2).

Purpose:

- Extracts image features using a pre-trained model.
- Organizes these features dynamically into a structured format for further analysis or machine learning tasks.

7.3 Merging and Saving Processed Data

```
# Merge the features DataFrame with the original tabular data
merged_data = pd.merge(df_cleaned, all_features_df, on='US_Number', how='left')

# Save the merged data to a new CSV file
merged_data.to_csv(to_save, index=False)

print("Feature extraction and merging completed. The results are saved in 'merged_data_with_image_features.csv'.")
```

7.3.1 Merge DataFrames:

- Combines the original tabular data (df_cleaned) with the image features DataFrame (all_features_df) using a left join on the US_Number column.
- Ensures all rows from the tabular data are retained, with matching image features added as new columns.

7.3.2 Save the Combined Data:

- Saves the merged dataset as a CSV file (to_save path), with rows representing individual records and columns for tabular data and dynamically named image features (e.g., Image_1_Feature_1, Image_2_Feature_1, etc.).

7.3.3 Confirmation Message:

- Prints a message to indicate successful completion of feature extraction, merging, and saving.

7.3.4 Result:

- A CSV file (merged_data_with_image_features.csv) containing:
- Original tabular data columns (e.g., demographics, lab results).
- Extracted image features for each US_Number.
- This file is ready for further analysis or modeling.

7.4 Cleaning Data by Dropping Columns with Excessive Null Values:

```
# Calculate the threshold for 60% null values
threshold = len(df) * 0.6

# Drop columns where the number of null values exceeds the threshold
df_cleaned = df.loc[:, df.isnull().sum() <= threshold]

# Display the shape of the cleaned dataframe
print("Original shape:", df.shape)
print("Cleaned shape:", df_cleaned.shape)
```

Original shape: (760, 38957)

Cleaned shape: (760, 4141)

7.4.1 Define a Threshold:

- Calculates the threshold for null values as 60% of the total number of rows in the dataset (threshold = len(df) * 0.6).
- Drop Columns Exceeding the Threshold:
- Removes columns where the number of null values is greater than the threshold (df.isnull().sum() <= threshold).
- Retains only the columns with a sufficient proportion of valid data.

7.4.2 Display DataFrame Shapes:

- Prints the original shape of the dataset (before cleaning) and the cleaned shape (after removing columns).

Purpose:

- Ensures the dataset is cleaned by removing columns with excessive missing values (>60%).
- Retains only useful columns for further analysis or modeling.

Modeling

8 Random Forest

8.1 Preprocessing and Selecting Data for Random Forest

```
##Random Forest
all_columns = df.columns.tolist()

categorical_features = ['Sex', 'Migratory_Pain', 'US_Performed', 'Ipsilateral_Rebound_Tenderness', 'Lower_Right_Abd_Pain', 'Coughing_Pain', 'Nausea']
numerical_features = [ 'Alvarado_Score', 'Body_Temperature', 'Paedriatic_Appendicitis_Score', 'WBC_Count', 'Neutrophil_Percentage', 'CRP', 'BMI', 'Height', 'Weight'] +
[col for col in all_columns if col.startswith('Image')]

# Preprocessing pipeline
preprocessor = ColumnTransformer(
    transformers=[
        ('num', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='mean')),
            ('scaler', StandardScaler())
        ]), numerical_features),
        ('cat', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='most_frequent')),
            ('ordinal', OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1))
        ]), categorical_features)
    ])
# Apply preprocessing
X_tabular = preprocessor.fit_transform(df.drop(columns=['US_Number', 'Diagnosis']))
y = df['Diagnosis'] # Assuming 'Diagnosis' is the target

# Now X_tabular is the preprocessed features and y is the target

pd.set_option('display.max_columns', None) # Show all columns
pd.set_option('display.max_rows', None) # Show all rows
pd.set_option('display.width', None) # Set display width to auto-fit
pd.set_option('display.max_colwidth', None) # Show full content of each column
# Select required columns with numerical first, then categorical
selected_columns = ['US_Number', 'Diagnosis'] + numerical_features + categorical_features

# Select 3 rows where Diagnosis is 1 and 3 rows where Diagnosis is 0
df_selected = pd.concat([
    df[df['Diagnosis'] == 'appendicitis'].head(4), # First 3 rows with Diagnosis == 1
    df[df['Diagnosis'] == 'no appendicitis'].head(4) # First 3 rows with Diagnosis == 0
], axis=0)

# Print the selected columns
(df_selected[selected_columns])
```

8.1.1 Define Features and Target:

Categorical Features: Includes binary or discrete columns such as 'Sex', 'Migratory_Pain', and 'US_Performed'.

Numerical Features: Includes continuous columns such as 'BMI', 'WBC_Count', and dynamically generated image feature columns (columns starting with 'Image').

8.1.2 Preprocessing Pipeline:

- **Numerical Features:**
 - Missing values are imputed using the mean.
 - Standardized using StandardScaler.
- **Categorical Features:**
 - Missing values are imputed using the most frequent value.
 - Encoded using OrdinalEncoder, where unknown categories are replaced with -1.

8.1.3 Apply Preprocessing:

- **Features (X_tabular):**
 - a. Applies preprocessing to all features, excluding 'US_Number' and 'Diagnosis'.
- **Target (y):**
 - a. The column 'Diagnosis' is used as the target variable.
 - b. Adjust Data Display Settings:
 - c. Modifies pandas options to display all rows and columns for better visibility.

8.1.4 Select a Subset of Data:

- Selects 4 rows where 'Diagnosis' == 'appendicitis' and 4 rows where 'Diagnosis' == 'no appendicitis'.
- Creates a small, balanced sample for display and debugging.

8.1.5 Output:

- Displays selected columns (selected_columns) from the sampled rows for review.

Purpose:

- Prepares data for Random Forest classification by handling missing values and scaling.
- Displays a sample of the preprocessed data for easy inspection.

8.2 Training and saving a Random Forest model

8.2.1 Display Feature Names

- Prints the **feature names** used during training, as processed by the preprocessor.

8.2.2 Split Data

- Splits the data into **training (80%)** and **testing (20%)** sets to evaluate the model.

Ensures reproducibility with random_state=42.

8.2.3 Encode Target Labels

Converts non-numeric target labels (y_train, y_test) into numeric codes using LabelEncoder, required by machine learning models.

8.2.4 Train a Random Forest Model

RandomForestClassifier:

- n_estimators=100: Uses 100 trees.
- class_weight={0: 1, 1: 2}: Assigns more weight to class 1 (e.g., "appendicitis") to handle class imbalance.

- `random_state=42`: Ensures reproducible results.
- **Training**: Fits the Random Forest model on the training data (`X_train`, `y_train`).

8.2.5 Predict and Evaluate

- **Predictions**: The model predicts labels (`y_pred`) for the test set (`X_test`).
- **Evaluation**: Prints a **classification report** with metrics like precision, recall, F1-score, and accuracy to assess model performance.

8.2.6 Save the Model and Preprocessor

- Saves the trained Random Forest model and the preprocessor pipeline as `.pkl` files for future use.
- `random_forest.pkl`: Contains the trained model.
- `preprocessor.pkl`: Contains the preprocessing steps applied to the data.

Purpose:

- Train a Random Forest model to classify the data effectively.
- Handle class imbalance using `class_weight`.
- Evaluate the model using the test set.
- Save the trained model and preprocessing pipeline for deployment or future predictions.

8.3 Output

	precision	recall	f1-score	support
0	0.78	0.83	0.81	95
1	0.69	0.61	0.65	57
accuracy			0.75	152
macro avg	0.73	0.72	0.73	152
weighted avg	0.75	0.75	0.75	152

➤ Class 0 (No Appendicitis):

- Precision (0.78): 78% of predicted no appendicitis cases were correct.
- Recall (0.83): 83% of actual no appendicitis cases were correctly identified.
- F1-Score (0.81): The harmonic mean of precision and recall.
- Support (95): 95 test samples belong to this class.

➤ Class 1 (Appendicitis):

- Precision (0.69): 69% of predicted appendicitis cases were correct.
- Recall (0.61): 61% of actual appendicitis cases were correctly identified.
- F1-Score (0.65): Represents the model's balance between precision and recall.
- Support (57): 57 test samples belong to this class.

8.4 Overall Model Performance

Metric	Value
Accuracy	0.75
Macro Avg (Avg of both classes)	0.73 (Precision), 0.72 (Recall), 0.73 (F1-score)
Weighted Avg (Weighted by class distribution)	0.75 (Precision), 0.75 (Recall), 0.75 (F1-score)

Accuracy (0.75): The model correctly classified 75% of test samples.

Macro Average: Simple average across both classes (useful for balanced classes).

Weighted Average: Adjusted for class imbalance (preferred when one class has more samples)

8.5 Visualizing Model Performance

```
matplotlib.use("TkAgg") # Or 'Qt5Agg', 'WXAgg', etc., depending on your system
# Confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, cmap='Blues', fmt='d')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```

```
# Visualize the tree
plt.figure(figsize=(20, 10))
plot_tree(tree, filled=True, feature_names=[f"Feature {i}" for i in range(X_tabular.shape[1])], class_names=["Class 0", "Class 1"])
plt.title("Decision Tree from Random Forest")
plt.show()
```

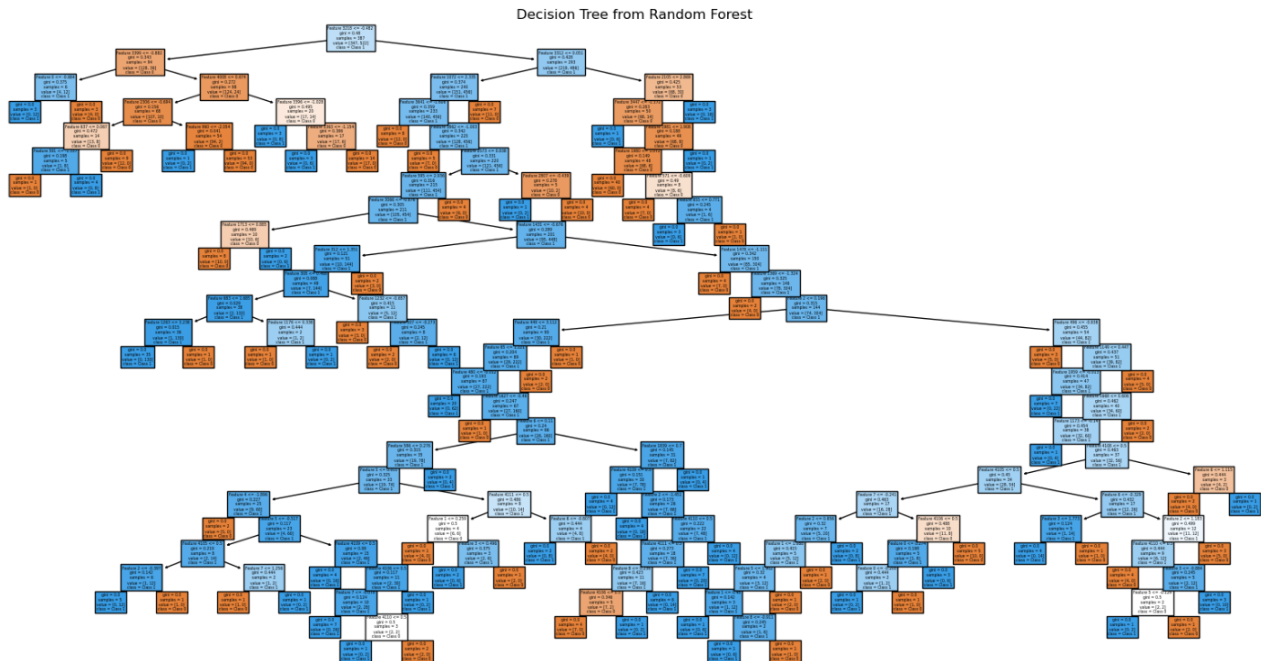
```
# Calculate accuracy using sklearn's function
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
```

8.6 Confusion Matrix Visualization

- **Purpose:** Displays the confusion matrix as a heatmap.
- **Rows:** Represent actual labels.
- **Columns:** Represent predicted labels.
- **Insight:** Helps assess how well the model classified each class.

8.6.1 Decision Tree Visualization

- **Purpose:** Visualizes one decision tree from the trained Random Forest model.
 - **Features:** Displays the features and thresholds used in the splits.
 - **Classes:** Highlights classification outcomes for each node.
1. **Accuracy Calculation**
 - **Purpose:** Calculates the model's accuracy using the test set.
 - **Insight:** Displays how many predictions were correct as a proportion of the total.



- **Confusion Matrix Heatmap:** Shows the distribution of correct and incorrect classifications.
- **Decision Tree Plot:** Explains how decisions are made in one of the trees from the Random Forest.
- **Accuracy Score:** Provides a numerical value for overall model performance.

9 Logistic regression

9.1 Encoding Categorical Features for Model Training

```
all_columns = df.columns.tolist()
categorical_features = ['Sex', 'BMI_Category', 'CRP_Category', 'WBC_Count_Category', 'Temperature_Category', 'Alvarado_Score_Category', 'Migratory_Pain', 'US_Performed', 'Ipsilateral_Rebound_Tenderness', 'Lower_RI']
numerical_features = ['Paediatric_Appendicitis_Score', 'Neutrophil_Percentage'] + [col for col in all_columns if col.startswith('Image')]
df_training = df[categorical_features + ['Diagnosis']]

# Create a LabelEncoder instance
label_encoder = LabelEncoder()

# Apply Label encoding to each categorical feature
for col in categorical_features:
    df_training[col] = label_encoder.fit_transform(df_training[col])
```

9.1.1 1. Define Features

- **Categorical Features:** Includes columns like 'Sex', 'BMI_Category', 'CRP_Category', and other categorical variables.
- **Numerical Features:** Includes 'Paediatric_Appendicitis_Score', 'Neutrophil_Percentage', and dynamically generated image features (columns starting with 'Image').

9.1.2 Create a Training Dataset

- Selects **categorical features** along with 'Diagnosis' (target variable) for encoding.

9.1.3 Apply Label Encoding

Converts categorical values into numeric labels:

- Example:

1. "Male" → 0, "Female" → 1
2. "High CRP" → 2, "Low CRP" → 1

Ensures categorical data is in a format suitable for machine learning models.

9.2 Training a Logistic Regression Model for Classification

```
X = df_training.drop(columns=['Diagnosis'])
y = df['Diagnosis']

# Encode categorical features and target
encoder = LabelEncoder()

# Apply label encoding to all categorical features
for col in categorical_features:
    X[col] = encoder.fit_transform(X[col])

# Encode the target
y = encoder.fit_transform(y)

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train the Logistic Regression model
model_logistic = LogisticRegression(random_state=42, max_iter=1000)
model_logistic.fit(X_train_scaled, y_train)

# Predict on the test data
y_pred = model_logistic.predict(X_test_scaled)

# Print classification report to evaluate the model
print(classification_report(y_test, y_pred))

pickle.dump(model_logistic, open("model_logistic.pkl", "wb"))

#model = pickle.load(open("random_forest.pkl", "rb"))
```

9.2.1 Define Features (X) and Target (y)

- X (Features): All columns except 'Diagnosis'.
- y (Target): The 'Diagnosis' column, representing the classification labels.

9.2.2 Encode Categorical Features and Target

- Encodes categorical features into numerical values.
- Encodes the target variable (y) into numeric labels for classification.

9.2.3 Split Data into Training and Testing Sets

- Splits dataset into 80% training and 20% testing.
- Ensures reproducibility with random_state=42.

9.2.4 Standardize the Features

- Standardizes numerical features to improve model performance.
- Ensures consistent feature scaling across training and testing sets.

9.2.5 Train a Logistic Regression Model

- Uses Logistic Regression for classification.
- Increases max_iter=1000 to ensure convergence during training.

9.2.6 Predict and Evaluate

- Predicts diagnoses (y_pred) using the trained model.

- Evaluates performance using a classification report (precision, recall, F1-score, accuracy).

9.2.7 Save the Model

- Saves the trained model as a .pkl file for later use.

Purpose

- Encodes categorical features for numerical processing.
- Splits and standardizes data to optimize model performance.
- Trains a Logistic Regression classifier for diagnosis prediction.
- Evaluates model performance and saves it for future use.

9.2.8 Output

	precision	recall	f1-score	support
0	0.79	0.73	0.76	95
1	0.60	0.68	0.64	57
accuracy			0.71	152
macro avg	0.70	0.71	0.70	152
weighted avg	0.72	0.71	0.71	152

➤ Class 0 (No Appendicitis):

- **Precision (0.79):** 79% of predicted **no appendicitis** cases were actually correct.
- **Recall (0.73):** 73% of actual **no appendicitis** cases were correctly identified.
- **F1-Score (0.76):** The harmonic mean of precision and recall.
- **Support (95):** 95 test samples belong to this class.

➤ Class 1 (Appendicitis):

- **Precision (0.60):** 60% of predicted **appendicitis** cases were correct.
- **Recall (0.68):** 68% of actual **appendicitis** cases were correctly identified.
- **F1-Score (0.64):** Represents the model's balance between precision and recall.
- **Support (57):** 57 test samples belong to this class.

Metric	Value
Accuracy	0.71
Macro Avg (Avg of both classes)	0.70 (Precision), 0.71 (Recall), 0.70 (F1-score)
Weighted Avg (Weighted by class distribution)	0.72 (Precision), 0.71 (Recall), 0.71 (F1-score)

- **Accuracy (0.71):** The model correctly classified 71% of test **samples**.
- **Macro Average:** Simple average across both classes (good when classes are balanced).
- **Weighted Average:** Adjusted for class imbalance (preferred when one class has more samples).

10 SVC

10.1 Training and Evaluating an SVM Model for Classification

```
from sklearn.svm import SVC
from sklearn.metrics import classification_report

y = df['Diagnosis']
X_train, X_test, y_train, y_test = train_test_split(X_tabular, y, test_size=0.2, random_state=42)
# Train the model using SVM
model_SVC = SVC(random_state=42)

# Fit the model on the training data
model_SVC.fit(X_train, y_train)

# Predict on the test data
y_pred = model_SVC.predict(X_test)

# Print classification report to evaluate the model
print(classification_report(y_test, y_pred))
```

10.2 Define Features (X_tabular) and Target (y)

- Target (y): The 'Diagnosis' column, representing the classification labels.

10.2.1 Split Data into Training and Testing Sets

- 80% training, 20% testing split.
- Ensures reproducibility with random_state=42.

10.2.2 Train an SVM Model

- Uses Support Vector Machine (SVM) for classification.
- Finds an optimal decision boundary to separate classes.

10.2.3 Predict and Evaluate

- Generates predictions (y_pred) on the test data.
- Evaluates model performance using a classification report (precision, recall, F1-score, accuracy).

Purpose

Trains an SVM model for diagnosis classification.

Evaluates the model using real test data.

Helps compare SVM performance with other models like Logistic Regression or Random Forest.

10.2.4 Output:

	precision	recall	f1-score	support
appendicitis	0.80	0.79	0.79	95
no appendicitis	0.66	0.67	0.66	57
accuracy			0.74	152
macro avg	0.73	0.73	0.73	152
weighted avg	0.74	0.74	0.74	152

10.2.4.1 Explanation of Each Metric:

- **Precision (Correct Predictions Among Predicted Positives)**
 - 0.80 for Appendicitis → 80% of the cases predicted as appendicitis were correct.

- 0.66 for No Appendicitis → 66% of cases predicted as no appendicitis were correct.
- **Recall (Model's Ability to Detect Actual Positives)**
 - 0.79 for Appendicitis → The model correctly identified 79% of actual appendicitis cases.
 - 0.67 for No Appendicitis → The model correctly identified 67% of actual no appendicitis cases.
- **F1-score (Harmonic Mean of Precision & Recall)**
 - 0.79 for Appendicitis → A strong balance of precision & recall.
 - 0.66 for No Appendicitis → Slightly lower, indicating some misclassification.
- **Support (Number of Samples per Class)**
 - 95 Appendicitis cases, 57 No Appendicitis cases.
 - Imbalanced dataset, with more appendicitis cases.

10.3 Model Performance & Evaluation

Each model was evaluated using **precision, recall, F1-score, and accuracy** based on appendicitis diagnosis.

Below are key observations:

10.3.1 Comparison of Model Performance

Model	Precision (Class 1)	Recall (Class 1)	F1-score (Class 1)	Overall Accuracy
Logistic Regression	0.60	0.68	0.64	71%
Random Forest	0.69	0.61	0.65	75%
SVC	0.66	0.67	0.66	74%

10.3.2 Key Insights:

- **Random Forest** had the **best overall accuracy (75%)**, leveraging ensemble learning for stability.
- **Logistic Regression** performed **worst in recall (0.68)**, indicating misclassification of some positive cases.
- **SVC** had balanced precision and recall, making it effective for small datasets.

11 Deployment Appendicitis Prediction Web Application

This deployment code utilizes **Flask** to create a web-based application for predicting the likelihood of appendicitis based on **tabular data and medical images**. The application integrates **machine learning models (Random Forest, Support Vector Classifier, and Logistic Regression)** along with a **pre-trained ResNet-50 deep learning model** for feature extraction from medical images. The key components and workflow of the deployment process are summarized below:

11.1 Initialization and Model Loading

- The Flask application is initialized (`app = Flask(__name__)`).
- Three machine learning models (Random Forest, SVC, and Logistic Regression) are loaded using **Pickle**.
- A **pre-trained ResNet-50** model (with the classification layer removed) is loaded to extract deep learning features from medical images.
- The **preprocessing pipeline** (scaling and encoding) is also loaded to ensure consistent feature transformation.

11.2 Image Feature Extraction Using ResNet-50

- A transformation pipeline (transforms.Compose) normalizes images to be compatible with ResNet-50.
- The `extract_features(image_path)` function reads and preprocesses uploaded images before passing them through ResNet-50 to obtain **feature vectors**.

11.3 Web Interface and User Input Handling

- The **Flask app provides an interface (index.html)** where users can upload **tabular medical data** (such as symptoms, lab test results) and **two medical images**.
- Routes (`/predict_random_forest`, `/predict_svc`, and `/predict_logistic_regression`) handle form submissions and process the input data.

11.4 Data Preprocessing and Feature Engineering

- Categorical variables (e.g., Sex, Migratory Pain, Nausea, etc.) are mapped to yes/no values.
- Numerical variables (e.g., Alvarado Score, Body Temperature, WBC Count, etc.) are extracted and converted into float values.
- Uploaded medical images are processed using ResNet-50 to extract deep learning features.
- The extracted **image features and tabular data** are combined into a single feature vector.

11.5 Model Prediction Process

- The combined data is transformed using a **preprocessing pipeline** (encoding and scaling).
- The **processed features** are then fed into the selected machine learning model (Random Forest, SVC, or Logistic Regression) for prediction.
- The application outputs one of the following results:
 - “**The patient is likely to have appendicitis.**”
 - “**The patient is unlikely to have appendicitis.**”

11.6 Error Handling and Deployment

- Exception handling ensures smooth execution and provides error messages in case of missing or incorrect input.
- The application runs on **port 5000** (`app.run(host='0.0.0.0', port=5000, debug=True)`) and allows remote access.

11.7 Conclusion

This deployment system effectively combines **machine learning (Random Forest, SVC, Logistic Regression)** and **deep learning (ResNet-50 feature extraction)** for **appendicitis diagnosis**. The integration of **tabular data and image processing** ensures comprehensive analysis, improving diagnostic accuracy. The application is user-friendly and accessible via a web interface, making it a practical tool for healthcare professionals.

12 User Interface

12.1 Random Forest

Appendicitis Prediction

Random Forest

SVC

Logistic Regression

Sex:	Alvarado Score:
Male	
Body Temperature (°C):	Pediatric Appendicitis Score:
WBC Count:	Neutrophil Percentage:
CRP:	BMI:
Height (cm):	Weight (kg):
US Performed:	Ipsilateral Rebound Tenderness:
Yes	Yes
Lower Right Abdominal Pain:	Coughing Pain:
Yes	Yes
Nausea:	Migratory Pain:
Yes	Yes
Upload Image 1:	
Choose File No file chosen	
Upload Image 2:	
Choose File No file chosen	
Predict with Random Forest	
The patient is likely to have appendicitis.	

12.2 SVC

Appendicitis Prediction

Random Forest **SVC** Logistic Regression

Sex: Male	Alvarado Score:
Body Temperature (°C): 	Pediatric Appendicitis Score:
WBC Count: 	Neutrophil Percentage:
CRP: 	BMI:
Height (cm): 	Weight (kg):
US Performed: Yes	Ipsilateral Rebound Tenderness: Yes
Lower Right Abdominal Pain: Yes	Coughing Pain: Yes
Nausea: Yes	Migratory Pain: Yes
Upload Image 1: Choose File No file chosen	
Upload Image 2: Choose File No file chosen	
Predict with SVC	
The patient is likely to have appendicitis.	

12.3 Logistic Regression

Appendicitis Prediction

Random Forest

SVC

Logistic Regression

Sex:	WBC Count:
Male	
CRP:	BMI:
Alvarado Score:	Body Temperature (°C):
Migratory Pain:	US Performed:
Yes	Yes
Ipsilateral Rebound Tenderness:	Lower Right Abdominal Pain:
Yes	Yes
Coughing Pain:	Nausea:
Yes	Yes

Predict with Logistic Regression

The patient is likely to have appendicitis.

13 Conclusion

In this study, Random Forest, SVC, and Logistic Regression were applied to predict appendicitis cases. Random Forest proved to be the most accurate and robust model for this task, while Logistic Regression offered interpretability, and SVC performed well with small, high-dimensional datasets. However, given the imbalance in the dataset, where 'no appendicitis' cases outnumber 'appendicitis' cases, handling this imbalance is crucial. Future improvements could involve combining these models to leverage their individual strengths and exploring techniques like oversampling or class weighting to address the class imbalance. Additionally, as clinical and image-based features are already integrated into the models, the next steps will include hyperparameter tuning, feature engineering, and integrating deep learning methods to further improve model performance and real-world applicability.